School of Computer Science and Information Technology
Lucerne University of Applied Sciences and Arts (Switzerland)

Development and Evaluation of an LLM-Based Code Assistant for IoT Payload Decoders

Bachelor Thesis

presented to the School of Computer Science and Information Technology of Lucerne University of Applied Sciences and Arts (Switzerland) in consideration for the award of the academic grade of *Bachelor in Business Information Technology.*

by

Tillo Beffa

from

Schwyz (Switzerland)

Declaration

Bachelor Thesis at Lucerne University of Applied Sciences and Arts
School of Computer Science and Information Technology

Title of Bachelor Thesis:	Development and Evaluation of an LLM-Based Code Assistant for IoT Payload Decoders
Name of Student:	Tillo Beffa
Degree Program:	Bachelor in Business Information Technology
Year of Graduation:	2026
Main Advisor:	Prof. Dr. René Meier
External Expert:	Robert Schlittler
Industry partner/provider:	Marius Giger, Akenza AG

Code/Thesis Classification

☒ Public (Standard)
☐ Private

Declaration

I hereby declare that I have completed this thesis alone and without any unauthorized or external help. I further declare that all the sources, references, literature and any other associated resources have been correctly and appropriately cited and referenced. All use of AI tools is documented in Appendix B.3. The confidentiality of the project provider (industry partner) as well as the intellectual property rights of the Lucerne University of Applied Sciences and Arts have been fully and entirely respected in completion of this thesis.

Place/Date, Signature _____

Expression of Thanks and Gratitude

I would like to take this opportunity to thank the entire team at Akenza for enabling me to work on such an interesting project. I would especially like to thank Marius Giger and Janick Rüegger for their mentorship, helpful guidance, insightful feedback, and constant availability. I am also grateful to Prof. Dr. René Meier for his valuable advice and for the time he dedicated to supervising this thesis.

Tillo Beffa, 2026

Intellectual property of the degree programs of the Lucerne University of Applied Sciences and Arts, FH Zentralschweiz, in accordance with Student Regulations: Studienordnung

Summary

Akenza is an IoT platform for connecting and managing a wide range of IoT devices and processing the data they generate. These IoT devices often send their measured data in compact, encoded payloads. To make this data usable for downstream data processing, it needs to be decoded and structured. For this, IoT platforms like Akenza usually develop a payload decoder. On the Akenza platform, a payload decoder is also called a device type and consists of multiple artifacts, including schemas, a JavaScript script, unit tests, and metadata.

Because many devices use a proprietary payload format, Akenza, which aims to support a large number of devices, needs many corresponding payload decoders. To address the time-intensive task of payload decoder creation, this thesis presents the development of an LLM-based assistant that streamlines this process by generating the key artifacts for a new payload decoder. To meaningfully assess and improve the assistant's performance, an evaluation benchmark was also created. This benchmark enabled an iterative, data-driven refinement cycle of the LLM-Assistant, which resulted in a relative increase of 23.4% in the benchmark score after a single cycle.

The practical value of the LLM-Assistant was quantified through a comparative case study, which demonstrated time savings of approximately 50% compared to the manual creation process. The assistant, while still a prototype, has already been successfully used in production by Akenza developers and in a customer project, confirming its real-world applicability.

Contents

1 Problem Definition and Vision **1**
- 1.1 Background and Problem Statement . . . 1
- 1.2 Thesis Goals and Core Objectives . . . 2
- 1.3 Stakeholder Relevance . . . 3

2 Background and State of Research **4**
- 2.1 IoT Payload Decoding . . . 4
- 2.2 Akenza Device Types . . . 6
- 2.3 LLM Capabilities & Application . . . 11
- 2.4 Benchmarking and Evaluation Methodologies . . . 13
- 2.5 LLMs in the Akenza Ecosystem . . . 14
- 2.6 Contribution of this Thesis . . . 15

3 Ideas and Concepts **16**
- 3.1 Research and Technology Exploration . . . 16
- 3.2 Development of an LLM-Assistant . . . 17
- 3.3 Development of an Evaluation Benchmark . . . 18
- 3.4 Iterative Refinement . . . 19
- 3.5 Quantifying Practical Value . . . 20

4 Methodology **21**
- 4.1 Qualitative Research Methods . . . 21
- 4.2 Design Science and Prototyping . . . 22
- 4.3 Quantitative Methods . . . 22
- 4.4 Project Management . . . 23

5 Implementation **24**
- 5.1 LLM-Assistant Prototype . . . 24
- 5.2 Custom Evaluation Benchmark . . . 32
- 5.3 Iterative Refinement Cycle . . . 40
- 5.4 Summary of Implementation . . . 43

6 Validation and Evaluation **44**
- 6.1 Case Study Setup . . . 44
- 6.2 Successful Deployments of the LLM-Assistant . . . 46
- 6.3 Refinement Cycle Validation . . . 46

6.4 Return on Investment Analysis . . . 46
6.5 Answering the Research Question . . . 48
6.6 Summary . . . 48

7 Outlook **49**
7.1 Areas of Improvement . . . 49
7.2 Conclusion . . . 50

Appendix **51**

A Supporting Materials **51**
A.1 Interview Transcripts . . . 51
A.2 Python Project Configuration . . . 56
A.3 Ground Truth Example . . . 57
A.4 Initial LLM System Instructions . . . 59
A.5 LLM-As-A-Judge System Instructions . . . 59
A.6 Artifact Requirements . . . 61
A.7 Test Set Device List . . . 63
A.8 Issues List From Evaluation Benchmark . . . 63
A.9 Issues List From Manual Inspection . . . 64
A.10 Notes Case Study Phases . . . 65
A.11 Problem Definition . . . 66

B Lists **72**
B.1 List of Figures . . . 72
B.2 List of Tables . . . 72
B.3 AI-Usage . . . 73

References **74**
References . . . 74

List of Abbreviations

ADK Agent Development Kit (Google).

AI Artificial Intelligence.

AJV Another JSON Schema Validator.

API Application Programming Interface.

AST Abstract Syntax Tree.

BLEU Bilingual Evaluation Understudy.

CLI Command-Line Interface.

CoT Chain-of-Thought.

DSR Design Science Research.

GPT Generative Pre-trained Transformer.

HTTPS Hypertext Transfer Protocol Secure.

IoT Internet of Things.

JDK Java Development Kit.

JSON JavaScript Object Notation.

LLM Large Language Model.

LoRaWAN Long Range Wide Area Network.

LPWAN Low-Power Wide-Area Network.

NN Neural Network.

OLAP Online Analytical Processing.

PoC Proof of Concept.

RAG Retrieval-Augmented Generation.

ROI Return on Investment.

SQL Structured Query Language.

TOML Tom's Obvious Minimal Language.

UI User Interface.

URL Uniform Resource Locator.

Chapter 1

Problem Definition and Vision

1.1 Background and Problem Statement

Akenza is an IoT platform for connecting and managing a wide range of IoT devices and processing the data they generate. These IoT devices often send their measured values in a very compact form, using an encoding such as hexadecimal (Leibbrandt, 2021). The data packages they send are called payloads and must be this compact to conserve transmission bandwidth and energy, which is particularly important for devices that are mostly battery-powered and operate over low-power networks like LoRaWAN (Casals, Gomez, & Vidal, 2023). However, for this raw data sent by devices, which is incomprehensible to humans, to become usable for downstream processing such as dashboard visualizations, alerts, analysis, or sending to external applications, it must first be interpreted and converted into a structured format, typically JSON (Akenza, 2025f). This necessary step usually involves creating so-called payload decoders. Creating payload decoders is a crucial yet technically challenging task, typically performed by Akenza, its users, or device manufacturers. Developing them requires dual expertise: an understanding of a device's unique payload format and the specific scripting logic and rules of the target IoT platform. There is a fundamental lack of payload standardization in the IoT industry. Nearly every device manufacturer implements a proprietary payload format for each specific device model (Noura, Atiquzzaman, & Gaedke, 2018). For IoT platforms like Akenza, supporting a wide variety of devices is one of the most important factors in attracting new clients. This creates the need for many custom payload decoders, which must be developed and maintained. Because creating even a single payload decoder can be time-consuming (Gutknecht, 2025), complex, and error-prone, there is a serious development bottleneck. This bottleneck establishes the core problem this thesis aims to address: the lack of an efficient, more automated solution for creating IoT payload decoders.

1.2 Thesis Goals and Core Objectives

The original problem definition of this thesis is provided in Appendix A.11.

1.2.1 Research Question

This thesis aims to create such a solution and to quantify its practical value. To guide the research and implementation, the following research question was formulated:

> *How, and to what extent, can an LLM-based system be used to assist and accelerate the development process in a domain-specific area such as IoT payload decoders?*

1.2.2 Core Objectives

To successfully answer the research question, the following five core objectives must be achieved:

1. **Research and Technology Exploration:** To investigate the state-of-the-art LLM capabilities in general and domain-specific code generation, prompt engineering strategies, and performance enhancement technologies such as Retrieval-Augmented Generation (RAG) and Agentic frameworks. Additionally, research different benchmarks and evaluate frameworks applicable in the field of domain-specific code generation.
2. **Development of an LLM-Assistant:** To design and develop a functional prototype of the LLM-based assistant, capable of generating key payload decoder artifacts, the decoder script, and its corresponding JSON schema.
3. **Development of an Evaluation Benchmark:** To design and implement an automated benchmark to assess the quality of the AI-generated code, based on metrics for syntactic correctness, completeness, and adherence to the scripting guidelines of the platform.
4. **Iterative Refinement:** To use the benchmark to evaluate the LLM-Assistant prototype and guide its refinement. The benchmark's results will be used to identify problem patterns, make changes, such as optimizing prompts or experimenting with different LLMs, and then reevaluate to confirm the refinement.
5. **Quantifying Practical Value and Efficiency Gains:** To establish a performance baseline for manual payload decoder creation and conduct a comparative case study against the LLM-Assistant's usage. The goal is to assess the tool's current and potential future impact on developer productivity and establish business value metrics such as time saved.

1.3 Stakeholder Relevance

The project is relevant and significant to both the business partner and the academic field.

1.3.1 Relevance for Akenza

Efficiency Boost: The goal of the thesis project is to make the development of payload decoders more efficient, ideally resulting in time and effort saved. With objectives 2-4, this goal is being attempted, and with Objective 5, these efficiency gains are being quantified.

Scalability and Onboarding: Streamlining the development of payload decoders will make the integration of new IoT devices into the platform easier and faster, making the platform more versatile and potentially attracting new clients.

1.3.2 Relevance for the Academic Field

The project will showcase how an LLM-enabled code assistant can be built and evaluated in a domain-specific context. It will also show how state-of-the-art LLM technologies, such as prompt engineering and agents, are used to solve a real-world problem.

Chapter 2

Background and State of Research

This chapter establishes the knowledge base for the thesis by reviewing and summarizing the current state-of-the-art for the topics IoT payload decoding, LLM capabilities & application in code generation, and methodologies for evaluating LLM-Generated code. To create a well-functioning and well-suited assistance system, it is also essential to consider the existing Akenza ecosystem in which the prototype will be built. For this reason, the relevant parts and processes of the Akenza platform will also be analyzed. The goal of this chapter is to identify existing knowledge, gain a better understanding of the project's context, pinpoint information that can be built upon, and highlight the research and knowledge gaps this thesis addresses.

2.1 IoT Payload Decoding

To understand the challenges an LLM-Assistant must overcome to make the development of IoT payload decoders more efficient, it is essential to first understand the domain of IoT payload decoding. The primary goal of IoT payload decoding is to translate the compressed and encoded payloads sent by an IoT device into a human-readable, structured format that is easier to work with in downstream data processing.

The need for IoT payload decoding stems from the way Low-Power Wide-Area Networks (LPWANs) like LoRaWAN, which many IoT devices use for networking and communication, operate. In LPWANs, it is necessary to keep data packets sent in the network as small as possible to conserve not only the device's battery life but also network bandwidth (Casals et al., 2023). For these reasons, device manufacturers design and program the device's firmware so that it transmits its captured data in a compact binary payload, typically represented as a hexadecimal string.

```
{
  "payloadHex": "01010124067 01A9E000000"
}
```

Figure 2.1: Example of a hexadecimal payload string (Akenza, 2025b)

```
{
  "temperature": 16.48,
  "humidity": 68.14,
  "lowTemperatureAlarm": false,
  "highTemperatureAlarm": false,
  "lowHumidityAlarm": false,
  "highHumidityAlarm": false
}
```

Figure 2.2: Example of the decoded payload (Akenza, 2025b)

The domain of IoT payload decoding lacks standardization. While standards like the LoRaWAN link-layer specification define the overall communication protocol (LoRa Alliance, 2020), there is no widely adopted universal standard for the application-layer payload format (Noura et al., 2018). Each device manufacturer, and often each device model, implements its own specific payload data format. The source of truth for interpreting and decoding this data is the device's decoding logic document, usually a PDF published on the manufacturer's website, which typically specifies:

- **Byte Layout:** A table defining which byte or sequence of bytes corresponds to a specific device data field/value.
- **Data Types:** The format of each value, for example, an unsigned 16-bit integer, with endianness byte ordering.
- **Bit Fields:** It's often the case that a single byte can contain multiple pieces of information, such as numerous status or alarm flags, each of which is encoded as individual bits and requires bitwise operations to extract.
- **Scaling and Offsets:** Raw integer values requiring conversion into real-world units via mathematical formulas. For example, to calculate the temperature in Celsius, the following formula needs to be applied: $T = \texttt{raw_value} \times 0.01$.

To decode data automatically, it is common practice to create a payload decoder. A payload decoder is platform-specific and usually consists of a script that implements device-specific decoding rules. This decoder script is the most critical part of the decoding process, as it automatically translates raw, compressed data into a human-readable, structured format (often JSON) (Leibbrandt, 2021).

For this thesis, the Akenza-specific rules and requirements for a payload decoder are relevant and will thus be further explained.

2.2 Akenza Device Types

On the Akenza platform, a payload decoder is called a device type. This section details the components that make up a Akenza device type and outlines the manual process for its creation.

2.2.1 Components of an Akenza Device Type

It is important to have a good understanding of all the device type components to recognize challenges and opportunities in the development process.

Decoder Script

The decoder script (`uplink.js`) itself is the core component of a device type. It is a script that must be written in JavaScript following ECMAScript 2020 (ES11). It must be self-contained, meaning it cannot rely on external libraries. It is, however, possible to use predefined helper functions, which are appended to the script at runtime (Akenza, 2025c).

On the Akenza platform, these self-contained decoder scripts are executed whenever a device payload is received by an Akenza data-gateway or broker. For context, the interval at which a device sends data can vary significantly depending on the device and use case, ranging from seconds to hours, with 5-15 minutes being the most common for a device sending over LoRaWAN (Tektelic, n.d.). To prevent data loss and enable load balancing, message brokers are installed before and after a Java microservice that runs the decoder script and captures its structured JSON output. The Java microservice is built on the GraalVM JDK, using its polyglot capabilities to execute JavaScript code (Akenza, 2025e) (GraalVM Project, 2025). Figure 2.3 showcases a high-level overview of the Akenza data processing pipeline.

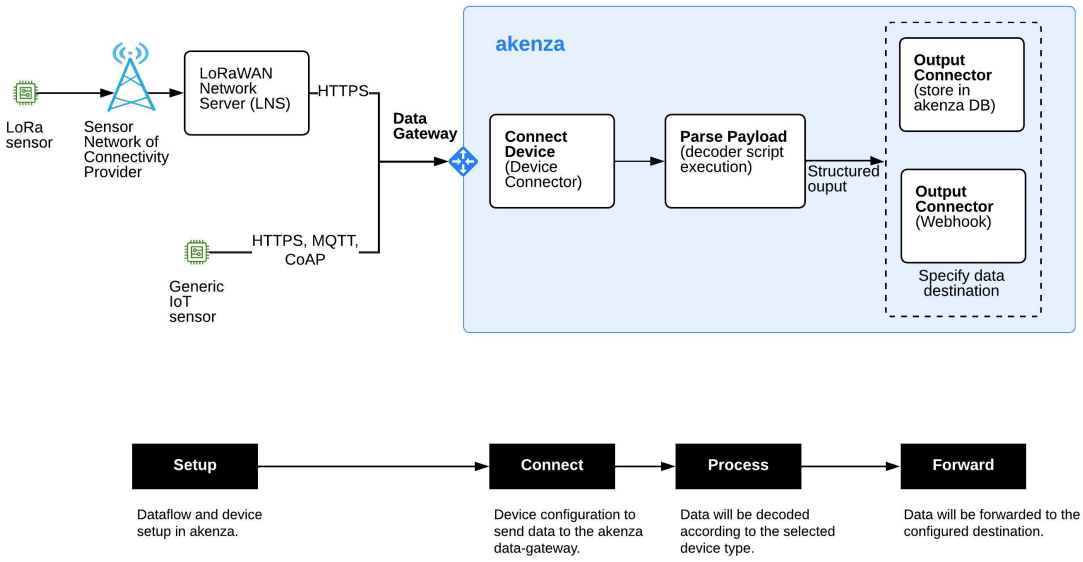


Figure 2.3: Akenza Data Processing Pipeline Simplified

The most important rules a decoder script needs to follow to work on the Akenza platform are the following (Akenza, 2025f):

- **consume() function:** Every script must implement a `consume(event)` function. This is the entry point for the script and will be invoked upon execution. This function receives an `event` object as an argument, which contains the raw hex payload and additional data, such as the transmission port.

- **emit() function:** To emit data, the decoder script needs to call the `emit()` function. This function is mocked in the GraalVM runtime to capture the passed arguments. The function accepts two arguments: the first defines the type of data being emitted ("sample", "log", "state", or "downlink"), and the second is an object with a predefined structure containing the data to be emitted. The `emit` function can be called multiple times per script run, resulting in multiple outputs.

- **Additional Rules:** Other constraints include a ban on `console.log` statements, JavaScript syntax extensions, and any file-loading functionality.

The following JavaScript code shows a sample decoder script emitting temperature and humidity data (Figure 2.4). Real decoder scripts can exceed a thousand lines of code; examples are available in the Akenza device type library on GitHub [1].

[1]Link to GitHub: `https://github.com/akenza-io/device-type-library/tree/main/types`

```
// script entry point
function consume(event) {
    // payload extraction
    const payload = Hex.hexToBytes(event.data.payloadHex);
    const defaultTopic = {};

    // decoding logic
    let temperatureValue = (payload[0] << 8) | payload[1];
    defaultTopic.temperature = temperatureValue / 100;

    const humidityValue = (payload[2] << 8) | payload[3];
    defaultTopic.humidity = humidityValue / 100;

    // data emit
    emit('sample', { data: defaultTopic, topic: 'default' });
}
```

Figure 2.4: Example uplink.js decoder script

Next to the decoder script, each device type requires additional components for reliable, well-documented decoding.

Topic Schemas

Because an IoT device usually sends different kinds of data, the decoder script splits it into categories. These categories of data are called topics. Normally, a payload contains data from at least two different topics: the measured data from its sensors (e.g., `temperature`) and device information (e.g., `batteryVoltage`). Some device types have upwards of ten topics. Each topic in a device type must include a corresponding topic schema (`*.schema.json`), which is a JSON Schema file used to validate the data emitted by a decoder script to this topic. The topics are not defined by the device manufacturer but by Akenza during device type development. The schemas help to quickly identify if something is wrong with the decoder script or the device data. A topic schema needs to follow JSON Schema Draft-07 (JSON Schema, 2017). To promote reusability within the topic schemas, Akenza maintains predefined property schemas for common data points, such as temperature, humidity, etc., which can be referenced via links to a central GitHub repository rather than being redefined in every topic schema. The naming of topics and their properties has additional rules which need to be followed (Akenza, 2025d). Figure 2.5 shows an example topic schema which would work with the decoder script from Figure 2.4.

```
{
  "$id": "https://akenza.io/netvox/device_name_example/default.
   schema.json",
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "processingType": "uplink_decoder",
  "topic": "default",
  "title": "Default",
  "properties": {
    "humidity": {
      "$ref": "https://raw.githubusercontent.com/akenza-io/
   device-type-library/main/data-models/environment/schema.json
   #/$defs/humidity/percent"
    },
    "temperature": {
      "$ref": "https://raw.githubusercontent.com/akenza-io/
   device-type-library/main/data-models/environment/schema.json
   #/$defs/temperature/celsius"
    }
  },
  "required": ["humidity", "temperature"]
}
```

Figure 2.5: Example topic schema (default.schema.json)

Metadata

Every device type must include device metadata (`meta.json`) as a JSON file containing information about the device name, manufacturer, device topics, device type version, device URL, and a list of onboarded sensors, etc. As an example, Figure 2.6 shows what the `meta.json` for the device Netvox R711 looks like.

```
{
  "name": "R711",
  "version": "1.0.0",
  "manufacturer": "Netvox",
  "url": "http://www.netvox.com.tw/product.asp?pro=R711",
  "description": "R711 is mainly used to detect the temperature
   and humidity of indoor air.",
  "author": "Akenza AG",
  "firmwareVersion": "V1.0.0",
  "loraDeviceClass": "A",
  "availableSensors": ["Temperature", "Humidity"],
  "outputTopics": ["default", "lifecycle"],
  "encoding": "HEX",
  "connectivity": "LORA"
}
```

Figure 2.6: Example meta.json for the Netvox R711 device (Akenza, 2024)

Unit Tests

To ensure reliability and prevent errors, each device type must include unit tests (`uplink.spec.js`). The unit tests are used to test decoder script functionality and schema adherence using sample payloads. These tests must pass before a device type can be published on the platform.

In summary, a complete device type for the Akenza platform consists of an `uplink.js` decoder script, a JSON schema for each topic `*.schema.json`, a `meta.json` file containing metadata, and an `uplink.spec.js` test file (Akenza, 2025f).

2.2.2 The Manual Device Type Creation Process

Three groups create new device types for the Akenza platform: Akenza developers, Akenza clients, and device manufacturers. It is planned that the assistance system will initially be used by just Akenza developers. This section details the creation process from their perspective. The process is not strictly standardized but generally begins when a new device needs to be integrated into the platform, often triggered by the need for new devices by an existing client or a new client onboarding. A crucial prerequisite for the development of a new device type is the availability of sample payloads for that device, either from the manufacturer or from an actual device already connected to the Akenza platform. These samples are essential for testing the functionality of the decoder scripts. Once it is confirmed that sample payloads are available, the typical creation steps are as follows:

- **Information Gathering:** The developer starts by collecting information, with the manufacturer's device decoding logic document being the most critical. This document typically explains the payload format and decoding logic, providing sufficient information to begin developing the decoder script. In some cases, manufacturers offer generic decoder scripts for their devices, which can serve as a starting point and just need to be adjusted to meet all Akenza platform requirements (Gutknecht, 2025).

- **Implementation and Testing:** While the selection of topics and properties is very important during the decoder script creation, the actual topic schemas are usually created after an initial decoder script has been written. After all the topic schemas are created and the decoder script is potentially adjusted again, the `meta.json` file is created. The implementation concludes with writing the `uplink.spec.js` file, which contains unit tests that verify the decoder script against the sample payloads (Gutknecht, 2025).

- **Code Review:** Before a decoder can be published on the Akenza platform, it must undergo a formal code review via a GitHub pull request.

The time required to create a complete device type varies significantly. It can

range from one hour to a full working day, depending on the complexity of the device, the quality of the decoding logic document, the availability and quality of any generic manufacturer-provided decoder script, and the developer's experience (Gutknecht, 2025).

2.3 LLM Capabilities & Application

This section outlines the capabilities and limitations of LLMs for code generation.

Large Language Models (LLMs) are a class of deep learning models that use the Transformer architecture and are usually trained on large amounts of text data. They operate by repeatedly predicting the next token (a sequence of characters often representing a word or part of a word) in a sequence. Through training on these vast datasets, LLMs are capable of generating human-like text in response to a given input sequence (prompt) (Stryker, 2024). Because the training data for these models also contains billions of lines of code, some models develop strong capabilities in generating code in addition to human language.

The field of code generation is led by proprietary models like OpenAI's GPT series, Anthropic's Claude series, and Google's Gemini series, as well as open-source alternatives such as Meta's Llama family or DeepSeek's models (Jiang, Wang, Shen, Kim, & Kim, 2024).

These models perform well on general programming problems and benchmarks, such as HumanEval, which primarily consists of programming contest problems (Chen et al., 2021). Their effectiveness decreases significantly when applied to specialized tasks, such as utilizing domain-specific libraries (Gu et al., 2025).

A study conducted by Shanghai Jiao Tong University (Gu et al., 2025) confirms that general-purpose LLMs struggle in such scenarios because their training data lacks sufficient examples of these niche or proprietary systems. The study lists the following as the most common failure modes:

- **API Misuse:** This is the most common issue identified, where the code generated by a model calls a real API function but with incorrect arguments or in the wrong sequence (Gu et al., 2025).
- **Missing API Calls:** The second most common problem is missing API calls, where the code generated by a model omits a necessary API call, resulting in a deviation from the expected functionality (Gu et al., 2025).

2.3.1 Instruction and Grounding Strategies

To address the challenges of domain-specific code generation, researchers have developed several strategies, including Prompt Engineering, Retrieval-Augmented

Generation, and Agentic approaches. These techniques focus on guiding the model with better instructions and grounding it with relevant, timely information.

Prompt Engineering

Prompt engineering is the practice of designing the input to an LLM to generate a better and more relevant answer (prediction) (Schulhoff et al., 2025). Recently, much research has been conducted on prompt engineering strategies. The following list contains some of the most popular and effective techniques:

- **Chain-of-Thought (CoT) Prompting:** This technique instructs the model to break down a complex problem into smaller steps and to "think out loud" before providing the final answer. Research shows that this approach improves the structure and quality of the output (Gu et al., 2025).
- **Few-Shot Learning/Prompting:** By including a few input-output examples in the prompt, the LLM has additional information about the desired prediction format and relevant context. This technique has been successfully used for teaching the model proprietary patterns without expensive fine-tuning (Schulhoff et al., 2025).
- **Structured Output Instructions:** This technique can be used to instruct the model to return its response in a specific format, such as JSON. This is beneficial when, for example, the goal is to generate JSON content, and also for use cases where LLM responses are subsequently used programmatically (OpenAI, 2024).

Retrieval-Augmented Generation (RAG)

RAG is a framework that tries to overcome the knowledge limitations of LLMs. Instead of relying solely on their static training data for knowledge, a RAG system dynamically retrieves relevant information from external sources, such as databases. It then injects (attaches) the retrieved information into an LLM prompt. For code generation, this process could involve indexing platform documentation, API specifications, and existing code examples in a database. At runtime, the relevant information is retrieved and attached alongside the user input, essentially grounding the LLM with up-to-date and relevant information (Gao et al., 2024).

LLM Agents

LLM agents are artificial entities that represent an approach to using LLMs in a more autonomous way. The first key feature of an LLM agent is the ability to make decisions. This is done by calling an LLM with a custom system instruction. The second key feature of an agent is its ability to use tools (take action). Tools are

functions that can be fixed, meaning no input is generated by an LLM, or functions where the input arguments are generated by an LLM. For example, when retrieving data from an SQL database, the agent can write the SQL query itself (Xi et al., 2023). The primary goal of LLM agents is to solve more complex tasks with better accuracy.

For most use cases, it makes sense to have multiple LLM agents, each with its own instructions and "expertise" (OpenAI, 2024). These agents can call upon one another in either a deterministic, fixed sequence or in a more non-deterministic manner, where agents can decide for themselves if a call to another agent is needed. To manage multiple agents, an agent framework is most often used, where agents can be easily configured and provided with new tools (Google, 2025a).

2.4 Benchmarking and Evaluation Methodologies

Evaluating the quality of generated code in an automated manner can be a complex task that varies a lot from use case to use case. This section reviews evaluation and benchmarking methodologies, as well as their limitations.

The standard for benchmarking the capabilities of LLMs for code generation was largely set by benchmarks like HumanEval, which tests functional correctness on self-contained programming problems using a metric called pass@k (Chen et al., 2021). This metric checks if at least one out of k generated code samples passes all unit tests. Newer alternatives, such as ClassEval, shift the initial generation task from simple self-contained problems to whole class-level generation, but still use unit tests in combination with pass@k for the actual evaluation (Du et al., 2023). The same goes for another modern code generation benchmark, EvoCodeBench; it frequently updates its programming tasks to ensure they are not in the models' training data, but also uses the pass@k metric in addition to recall@k, which measures correct identification and usage of internal tools and functions (Li et al., 2024). While pass@k is the de facto standard for evaluating LLM code generation, it has several limitations. It struggles particularly with tasks that have multiple valid answers or tasks that require following specific coding rules. The main problems are as follows:

- **Binary Success Metric:** The pass@k metric is all-or-nothing. It fails to capture partial correctness, where a nearly perfect solution might still be useful to a developer but is marked as a total failure because of a minor error (Paul, Zhu, & Bayley, 2024).

- **Rigidity of Unit Tests:** Unit tests require the code to return a result in predefined format. This is a problem when a wide range of outputs could be considered correct. It is often very hard to write a unit test that accounts for every possible variation of a correct answer.

- **Lack of Domain-Specific Rules:** The pass@k metric focuses only on

whether the code works (functional correctness). It does not check if the code follows specific guidelines or proprietary rules that are often required in domain-specific areas.

Before the HumanEval benchmark popularized the pass@k metric, the BLEU score was frequently used for LLM evaluation. BLEU is a text-matching metric that compares a generated sample to a reference solution using n-gram overlaps (Papineni, Roukos, Ward, & Zhu, 2002). However, it is no longer widely used for code evaluation due to its inability to assess functional correctness; code often has many valid syntactic forms that produce the same correct output, which simple text-matching fails to capture. CodeBLEU was developed to address these limitations specifically for LLM-generated code. It extends beyond lexical n-gram matching by additionally analyzing the code's syntactic structure, via Abstract Syntax Trees (AST), and data flow (Ren et al., 2020).

Another promising technique for evaluating LLM-generated code is LLM-as-a-judge. Frameworks like CodeJudge use an LLM to perform a step-by-step analysis to determine code correctness without needing predefined test cases (Tong & Zhang, 2024). However, using LLMs to judge LLM-generated code brings its own challenges, such as mitigating hallucinations where LLMs generate plausible but incorrect code, like calling non-existent API functions, which are hard for an LLM judge to flag as wrong (Huang et al., 2025; Crupi et al., 2025).

2.5 LLMs in the Akenza Ecosystem

The Akenza platform currently features one primary LLM application called Genio[2]. Although the LLM-Assistant built for this thesis will not initially be part of Genio it is still important to understand the existing AI/LLM landscape at Akenza to make sustainable, well-informed decisions about technology selection (Rüegger, 2025).

2.5.1 Genio

Genio is a chat interface that allows users to prompt an LLM about their IoT devices and the data stored on the platform. It aims to lower the entry barrier for data analysis and allows non-technical users to use their IoT data out of the box. For instance, instead of building a dashboard widget on the Akenza platform or manually filtering a device list, a user can simply ask Genio. In the backend, Genio acquires the relevant information with an agentic RAG approach and returns a response in natural language (Rüegger, 2025).

The development of Genio started in 2024, evolving from a simple multi-prompt LangChain application into a more sophisticated agentic system. This architec-

[2]Akenza - Genio: `https://akenza.io/genio`

tural shift was primarily driven by the need to serve more complex use cases. Genio currently uses Google's Agent Development Kit as its foundation. It contains multiple specialized agents, such as a health agent (for device-health-specific use cases) able to query a ClickHouse OLAP database, a measurements agent (to retrieve relevant device measurements) able to access Akenza's API, a data fusion agent able to query a Neo4j graph over a GraphQL API, as well as a planner agent that dynamically creates a plan of which agents need to be invoked in what order to satisfy the user's query (Rüegger, 2025).

2.5.2 Evaluation Methods for Genio

To assess the performance of Genio, Akenza has created a manual evaluation process. This process is centered around a prompt library that contains real-world questions for various use cases a user might ask Genio. Periodically, these prompts are run through the Genio system, and the responses are captured, manually evaluated, and scored on a scale between 0 and 1. This manual approach is currently favored over an automated approach due to its simplicity and the inherent challenges that automating evaluation for such a dynamic system would bring. Genio's predictions are based on live, real-world data that is frequently updated. This makes reproducible unit tests, for example, difficult. Additionally, Genio uses Comet Opik as a debugging and monitoring solution to track agent behavior (Rüegger, 2025).

2.6 Contribution of this Thesis

The review of the state of research and the analysis of the Akenza platform provide a solid foundation of existing knowledge and reveal a clear set of gaps that this thesis aims to address.

The project will try to apply reviewed concepts such as prompt engineering, agentic frameworks, and principles for evaluating LLM-generated code, following scientific methods, all while leveraging Akenza's existing infrastructure and technologies.

Chapter 3

Ideas and Concepts

This chapter outlines the key ideas and concepts necessary to achieve the thesis's objectives and answer the research question. Each of the five objectives will have its own section with relevant ideas and concepts.

3.1 Research and Technology Exploration

The first objective, Research and Technology Exploration, is fundamental to the thesis because it establishes essential background knowledge and informs decisions on technology selection and architecture for subsequent objectives. The research is separated into two streams: an investigation of the state of the art and an analysis of the Akenza ecosystem and platform. The state-of-the-art research will follow a structured plan. Different topics were identified for which relevant papers and additional resources, such as blogs or documentation, will be read. The key findings will be summarized in Chapter 2.

LLM Capabilities & Application	**Evaluation Methodologies**	**IoT Payload Decoding**
Domain-Specific Code Generation	Generated-Code Benchmarks	LPWAN Communication Constraints
Prompt Engineering (CoT, Few-Shot)	Heuristic Metrics (e.g., BLEU, CodeBLEU, Levenshtein Ratio)	Binary Payload Encoding & Structure
Retrieval-Augmented Generation, Agent Frameworks	LLM-as-a-Judge	IoT Standardization

Table 3.1: State-of-the-art Research Plan Topics

At the same time, a thorough analysis of the Akenza ecosystem will be conducted

to ground the project in its context. This contextual understanding is essential for ensuring the proposed solution is relevant, technically feasible, and solves the proposed problem. The analysis will investigate how the relevant components and processes of the Akenza IoT platform function, including the components of a device type and the process of creating one, as well as the architecture and technologies employed in existing LLM products on the Akenza platform. This analysis will be executed by reviewing documents, including platform documentation, and conducting interviews with Akenza employees. The findings will also be summarized in Chapter 2, and the transcripts of the interviews are provided in Appendix A.1.

3.2 Development of an LLM-Assistant

The LLM-Assistant will be developed as a prototype, focusing on the primary functionality: making the process of payload decoder development more efficient by generating key artifacts, such as the decoder script and corresponding JSON schemas. The goal of the prototype is not to be a complete, ready-to-deploy system, but rather to provide a proof of concept.

3.2.1 Possible Architectures and Technologies

Python will be used as the programming language for implementing the majority of the prototype, primarily due to its large community and numerous libraries in the fields of RAG and agent frameworks (Eastgate Software, 2025), as well as because the existing LLM product from Akenza uses Python (Rüegger, 2025).

The prototype will be built iteratively, allowing its architecture and complexity to evolve. This approach should ensure the best possible outcome and allow for incremental testing of core functionalities. The goal of the LLM-Assistant should be to generate all the needed artifacts for an Akenza device type. These artifacts do not need to be perfect, as the LLM-Assistant is intended to be a helpful tool used alongside the development process rather than to replace it. The artifacts can then be used as a starting point for further adjustments and testing.

It is planned to divide the overarching LLM-Assistant goal into subtasks, with each subtask generating one of the artifacts that make up a full device type. This can be achieved either through multiple subsequent or parallel API calls or by using agents. Because Genio uses Google ADK as its agent framework (Rüegger, 2025), it would make sense to adopt the same technology for this prototype, ensuring easier integration of the LLM-Assistant into Genio at a later stage. The idea is to start simple with direct API calls and, if beneficial, later move to an agent architecture. The approach of not just generating all relevant artifacts with a single LLM call is based on research showing that LLMs perform more reliably when a prompt is focused on a single, well-defined goal (Yang et al., 2025).

The benefits an agent-based architecture can offer include error isolation and modularity: when a failure occurs, it can be pinpointed to a specific agent in the pipeline, simplifying debugging. Individual agents can be improved or replaced easily without requiring redesign of the entire system.

3.3 Development of an Evaluation Benchmark

The goal of this objective is to create a benchmarking framework that reliably evaluates the performance of the LLM-Assistant.

The benchmarking framework should also be developed iteratively to find the best balance between manual and automated evaluation.

As a first experiment, the benchmark could adopt the manual evaluation process used for Akenza's Genio assistant. This would involve creating a fixed dataset of device types, repeatedly generating artifacts for them, and manually scoring each artifact on a predefined scale (e.g., 0 to 1). This approach promises to be simple to implement but time-consuming and not very objective to execute.

An additional experiment could be to automate evaluation using unit tests, a method popularized by LLM benchmarks such as HumanEval and its pass@k metric (Chen et al., 2021). As discussed in the research chapter, this approach has known shortcomings. The binary pass/fail nature of unit tests is probably too rigid, especially since the decoder script and additional artifacts, which the LLM-Assistant tries to generate, as well as the decoder script's structured JSON outputs, have multiple valid forms.

Given that every existing decoder on the Akenza platform already has accompanying unit tests, it would still be worth a try to create a dataset of existing device types, regenerate the necessary artifacts with the LLM-Assistant, and run the unit tests on them. The results can then be compared against results from manual evaluation. The process should provide an overview of how meaningful unit test scores are and whether further time should be invested in refining them.

Because it is important not only to evaluate the final JSON output of a decoder script, but also the form of the decoder script and accompanying schemas, additional metrics could be calculated that indicate the LLM-Assistant's performance. Potential metrics for this benchmark include:

- **Syntactic Correctness:** Validating the generated JavaScript code and JSON schemas.
- **Platform Adherence:** Ensuring compliance with Akenza-specific rules (e.g., scripts must be self-contained).

3.3.1 Meta Evaluation

An evaluation benchmark that outputs a score is only useful if this score correlates with the actual quality of the evaluated artifacts. To ensure this, a meta-evaluation will be performed. This involves using different Large Language Models with varying coding capabilities in the LLM-Assistant to generate the same sets of artifacts. The sets of artifacts will then be evaluated both by the evaluation benchmark and by hand. A correlation between the benchmark's scores and manual assessment scores should then indicate the benchmark's meaningfulness.

3.4 Iterative Refinement

Once both the LLM-Assistant prototype and the evaluation benchmark are functional, this objective aims to establish a refinement cycle with data-driven refinements to increase the LLM-Assistant's performance. The following is an approach for how the process could look:

1. **Device Selection:** Select multiple devices with varying complexity and create a test set.

2. **Generation:** The LLM-Assistant should generate device artifacts for each device in the test set.

3. **Evaluation:** The benchmark should evaluate the artifacts and return an overall performance score. If possible, the benchmark should also identify issues, such as incorrect syntax.

4. **Failure Pattern Analysis:** A set of LLM-Assistant issues should be created, consisting of the issues found by the evaluation benchmark and additional issues identified manually.

5. **Targeted Intervention:** Each issue found should drive a specific refinement. It is probably better to address one issue at a time and rerun the evaluation benchmark on a specific device in between to see whether the changes improved resolved the targeted issue. A Intervention could include adjustments to the agent invocation order, LLM adjustments (such as changing models or experimenting with different temperature and structured output settings), or adjustments to system prompts.

6. **Re-Evaluation:** After all planned interventions have been implemented, the complete evaluation benchmark with all devices from the test set can be re-executed to verify that the changes have resolved the targeted issues without introducing new ones and that the overall benchmark score has improved.

For this thesis, the plan is to do at least one complete data-driven refinement cycle that demonstrates that LLM-Assistant's performance can be improved using this data-driven approach.

3.5 Quantifying Practical Value

The LLM-Assistant can be considered a success if it demonstrates that it has solved the development bottleneck described in the problem definition by reducing the time required to develop a device type. This objective aims to measure the LLM-Assistant's actual impact on developer productivity, thereby enabling the calculation of the project's return on investment. To measure the impact of the LLM-Assistant, a comparative study approach will be used. The first part of the study will establish the baseline by examining the manual process for developing device types, while the second, LLM-enabled part will establish the comparative metric. The study will focus on the time saved.

Chapter 4

Methodology

This chapter explains the methods and approach used to answer the research question and achieve the thesis's core objectives. The project follows an iterative process model, which is well-suited for research and development where insights from one phase inform the next. The overall project roadmap, including key phases and milestones, is visualized in Figure 4.1.

To systematically answer the research question, this thesis uses a mixed-methods approach. It combines qualitative research to gain background knowledge, Design Science Research for artifact creation, and a quantitative method for validation.

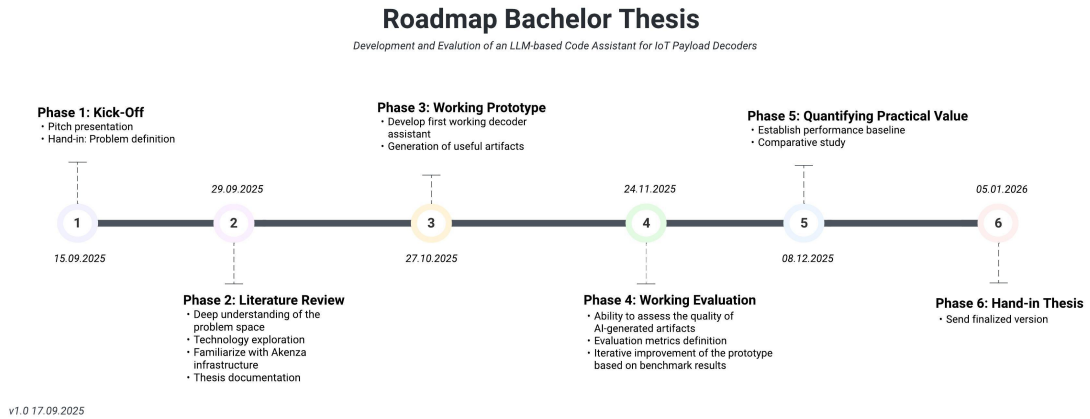


Figure 4.1: Roadmap Bachelor Thesis

4.1 Qualitative Research Methods

To establish theoretical and domain knowledge, the initial phase of the project relies on qualitative methods.

- **Literature Review:** A review of existing academic and industry literature

was conducted to identify the state-of-the-art in LLM capabilities and applications, evaluation methodologies, and IoT payload decoding. This ensured that the prototype and evaluation benchmark are built upon current best practices.

- **Domain Analysis:** To understand the problem domain, an analysis of the Akenza platform and processes was performed. This involved examining the manual device type creation process, as well as an exploration of the existing LLM-enabled tool, Genio. The domain analysis was performed by reviewing platform documentation and the codebase, as well as conducting employee interviews.

4.2 Design Science and Prototyping

The core of this thesis follows the Design Science Research (DSR) paradigm, which focuses on the creation and evaluation of artifacts to solve real-world problems.

- **Artifact Creation:** Two primary artifacts were developed: a functional prototype of the LLM-Assistant (Objective 2) and a custom evaluation benchmark designed to evaluate its outputs (Objective 3).

- **Iterative Development:** The prototype was refined through an iterative cycle (Objective 4). This process used the feedback from the benchmark to identify weaknesses and guide data-driven refinements to the LLM-Assistant by adjusting agent models and agent instructions.

4.3 Quantitative Methods

To validate the prototype's effectiveness and measure its impact, an additional quantitative method was used.

- **Comparative Study:** A comparative case study was conducted to quantify the practical value of the prototype (Objective 5) by measuring the time saved by the LLM-Assistant compared to the manual device type creation process. As the study was conducted with one participant, the results provide only an initial indication of the tool's potential impact on developer productivity and cannot be generalized. This estimated time saving then allowed for the creation of a return on investment projection for the whole project.

4.4 Project Management

To ensure structured progress and timely completion, the project was managed using an agile methodology. The work was organized into weekly cycles, with progress tracked on a Jira board. Daily stand-up meetings were held with the industry partner to discuss progress, address challenges, and align on next steps. All source code and documentation were version-controlled using GitLab. The bachelor thesis document itself was written throughout the project implementation in Overleaf.

Chapter 5

Implementation

This chapter describes how the ideas and concepts presented in Chapter 3 regarding the LLM-Assistant prototype, the evaluation benchmark, and the iterative refinement cycle were implemented. It highlights key decisions, technology and architecture choices, development steps, encountered challenges, and the limitations of the final artifacts.

5.1 LLM-Assistant Prototype

This section focuses on the LLM-Assistant and starts with an explanation of the technology stack. It then provides details on how the LLM-Assistant was built and describes its final architecture, concluding with a demonstration of how the LLM-Assistant can be used.

5.1.1 Technology Stack

The decisions regarding the technology stack were primarily based on the research conducted, the existing technology used in Akenza's Genio, and prior personal experience.

Programming Language and Libraries

Python was selected as the primary programming language for implementing the LLM-Assistant. This decision was based on the reasons provided in Section 3.2.1.

Astral's uv was used as the Python package and project manager. uv allows for easy package and virtual environment management. It was selected over pip due to its convenient features, such as a project-level TOML file, and its faster performance (Astral, 2025).

Different Python libraries, such as Google ADK, Click, and Pydantic, were used in the LLM-Assistant. As not all of these libraries have a significant impact on the LLM-Assistant, only those that do will be explained in a corresponding section. The complete `pyproject.toml` can be found in Appendix A.2.

LLM and Agent Framework

Google's Agent Development Kit (ADK) was selected as the backbone for all LLM-related tasks. This decision was heavily influenced by its current use in Akenza's Genio. Other frameworks, such as LangGraph and CrewAI, were also considered. Google ADK is provided as a Python library. No exclusive large language model or provider was chosen, as Google ADK allows for switching between them easily. However, because Google Gemini models offered additional features in combination with Google ADK, like easy tool integration, they were the primary models used (Google, 2025a). Since the LLM-Assistant never comes into contact with personal client information, the publicly available API endpoints were used and no private model instances were needed.

Paradigms

The code of the LLM-Assistant uses a combination of different programming paradigms. It uses object-oriented programming for all Google ADK-related classes, including agents, runners, and the memory service, as well as for classes representing a device type. The LLM-Assistant uses Python's `asyncio` library to handle concurrent operations, such as communication with LLMs when creating multiple device types at once. The generation workflow logic is written as a sequence of imperative steps, such as setting up sessions, processing events, and saving files.

5.1.2 Development Process

This section describes the development of the LLM-Assistant prototype, focusing on its iterative evolution, architecture, and core components.

System Evolution

Because the LLM-Assistant aims to solve the development bottleneck through LLM code generation, it was first necessary to define what an ideal assistant would look like. As discussed in Chapter 3, generating device type artifacts and thus minimizing manual labor would be ideal. The decision was made to start simple by testing LLMs through the providers' respective web UIs rather than building a full-fledged assistant. With the experience gained, the system was iteratively improved to be more automated and robust. Every decision for refinement was

guided by the experience of the previous version. All outputs from the LLM-Assistant during development were evaluated manually because the evaluation benchmark did not exist at that time.

First Experiments

To validate the artifact generation idea, a small system instruction was created with the goal of generating a payload decoder script using LLM providers' web UIs to interact with the models. It was tested on different LLMs (Google Gemini 2.5 Pro and ChatGPT-5).

The first version of the LLM system instruction can be found in Appendix A.4.

The results from this first experiment were promising. Upon manual inspection of the LLM outputs, it was clear that the models seemed to generate plausible decoder JavaScript code with mostly correct syntax. However, the decoder scripts were still far from perfect and had two major problems: (1) They did not follow any Akenza scripting rules. (2) The decoder scripts were inconsistent, meaning that for the same input, the LLMs generated different outputs. To make the generated decoder scripts work on the Akenza platform, a lot of additional manual changes were required. Still, the experiment was sufficient validation to further pursue the artifact generation idea.

Programmatic LLM-Assistant

To tackle the issues identified in the first experiment, the first version of the programmatic LLM-Assistant was developed. The LLM-Assistant now used API calls to communicate with LLM models, which enabled additional configuration of hyperparameters like temperature and structured output. The whole LLM-Assistant Python script also used Pydantic for API response validation. The system instructions were also adjusted to include the complete Akenza scripting documentation (Akenza, 2025c).

The new prompt design of the system instruction was heavily influenced by the paper (Schulhoff et al., 2025). It was now separated into two different parts:

- **Part 1: Static:** The main system instruction with role, goal, and output format, as well as a section for domain-specific knowledge (Akenza scripting rules).
- **Part 2: Dynamic:** Device name, manufacturer, and decoding logic.

The prompt was designed this way to make use of prompt caching to speed up generation while lowering the cost of input tokens (Sankalp, 2025).

The results from this version of the LLM-Assistant were significantly more consistent, mainly due to lowering the model temperature and using structured JSON

outputs. The domain knowledge provided directly within the system instruction resulted in decoder scripts that started to follow the desired script structure, implementing a `consume()` function as the script entry point and calling the `emit()` function to output data. However, the format of the emitted data had a major flaw: the generated scripts normally did not separate data into topics and used arbitrary property names.

Google ADK and Agent Adoption

Because a valid decoded payload data format is very important for an Akenza device type, the next version of the LLM-Assistant focused on adopting an agent architecture using Google ADK. The idea was to split the decoder logic creation and topic selection into subtasks performed by two separate agents: a schema agent and a decoder agent. Even though the manual process of creating a device type typically starts with the decoder script itself (Gutknecht, 2025), the decision was made to first generate topic schemas. These schemas act as a contract that the subsequent decoder agent must follow. The challenge was to make the schema generation accurate enough so that all data points sent by a device were first correctly identified and then assigned to a correctly chosen topic. If flawed topic schemas, for example, ones with missing properties, were generated, the decoder agent was likely to produce a script that also failed to emit the forgotten properties.

The two agents run sequentially. Google ADK offers different out-of-the-box options to handle agent state and context. The decision was made to feed the input and output of the first agent directly into the next agent. This was easy to implement and made sense because the decoder agent has a direct dependency on the schema agent's output.

5.1.3 Final LLM-Assistant Prototype

This section explains the final version of the LLM-Assistant. Before the LLM workflow can begin, a user initiates the assistant by inputting the manufacturer name, device name, and as much information about the decoding logic as possible via a command-line interface (CLI). The decoding logic can be provided in .txt and .pdf files.

With the user input in place, the LLM workflow starts. This final LLM-Assistant prototype consists of four agents. As before, the different agents are invoked in a predefined, sequential order. The following visualization shows the agent workflow.

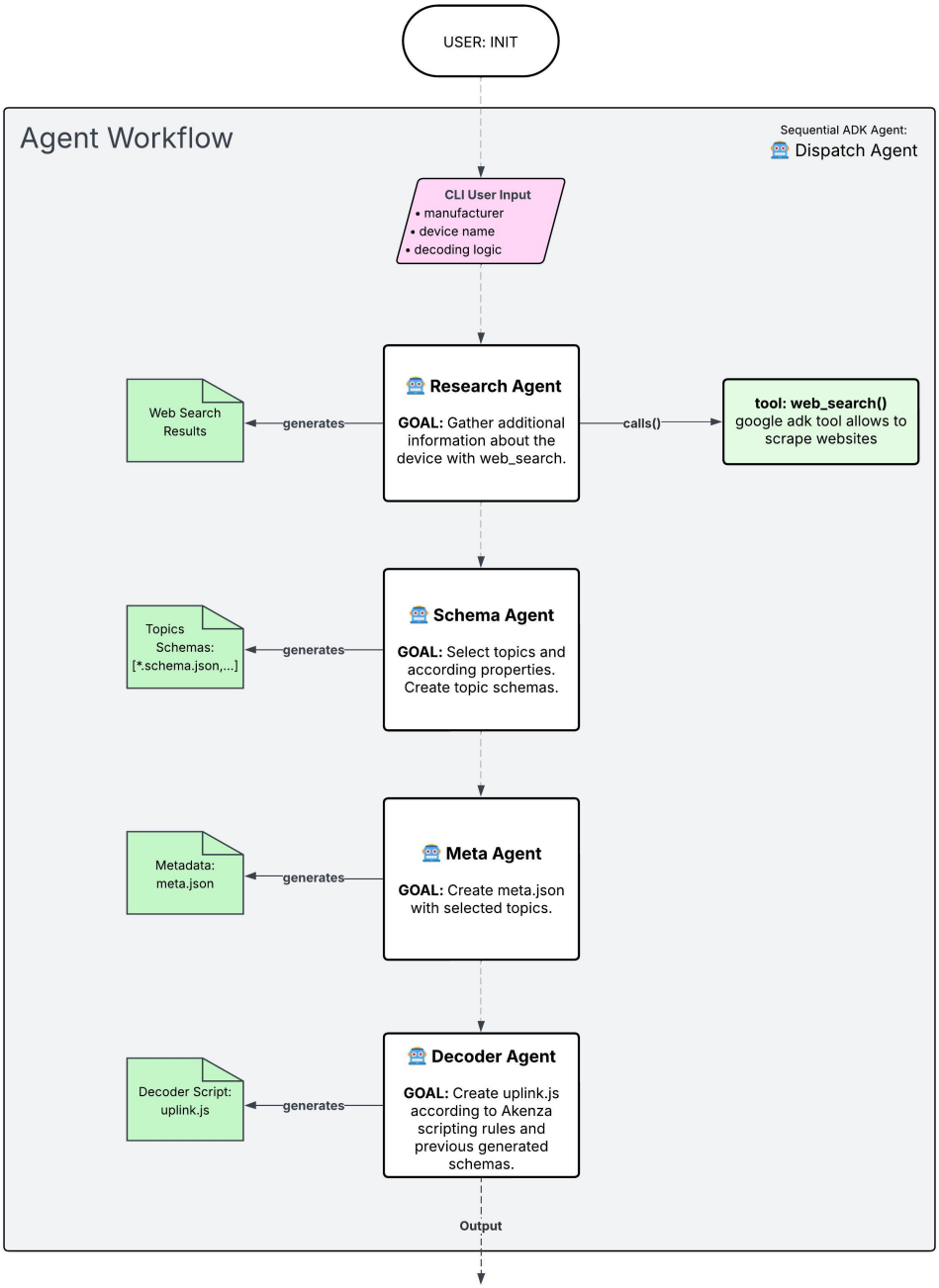


Figure 5.1: Agent workflow of final LLM-Assistant prototype

Explanation of Agents

Research Agent From the experience of the previous LLM-Assistant iterations, it was clear that the more relevant information the LLMs receive as input, the better their output tends to be. Before any device type artifacts start being generated, an additional research agent has been added that can call a web search tool to scrape websites. This agent is tasked with gathering additional relevant information about the device's decoding logic and finding generic payload decoder

scripts.

Schema Agent This agent is invoked after the research agent finishes and receives all user input, alongside the findings from the research agent. The goal of this agent is to select data categories (topics), assign all data points sent by a device to these topics as properties, and then create a topic schema. As mentioned in the research on the manual device type creation process in Section 2.2.1, not every property needs to be created from scratch. Many properties are already available on Akenza's GitHub[3] and simply need to be referenced. A list of all existing properties, including links for reference, is appended to the agent's system instruction. The system instruction also includes exemplars as well as schema creation rules. The agent's final output is a JSON object containing an array of topic schemas.

Meta Agent Because the decoder agent performs better when fed more relevant information, an additional meta agent was created to run after the schema agent. This agent is tasked with creating the `meta.json` artifact, which includes all the topics a device supports, as well as metadata such as the name, firmware version, manufacturer, device URL, and onboard sensors. As with the previous agents, this agent has access to all previously generated and entered data. The final output of this agent is a JSON object containing the `meta.json`.

Decoder Agent The final agent to run is the decoder agent, with the goal of generating the decoding script while adhering to all Akenza scripting rules and emitting data with a structure defined by the schema agent. This agent's system instruction has been extended again from the previous LLM-Assistant version to also include exemplars. This agent also returns a structured JSON object containing the JavaScript code.

All agents, except for the research agent, return their outputs as predefined, structured JSON objects, which allows for easier handling in subsequent processing. As soon as the agent workflow finishes, the subsequent processing starts by reviewing all agent events, extracting the JSON objects, formatting and parsing them into the desired structure, and finally saving them as files to disk. With this, the LLM-Assistant has finished and shuts down.

Model Choice

Google ADK allows for changing models on a per-agent basis. The LLM-Assistant has undergone a thorough manual evaluation, which resulted in the selection of the Gemini 2.5 Pro and Gemini 2.5 Flash models. For the research and meta agents, the Flash model is used, as it is faster and cheaper. Since these tasks are not very

[3]See Akenza Data Models: `https://github.com/akenza-io/device-type-library/tree/main/data-models`

complex, this model is a suitable choice. For the schema and decoder agent, the Pro model yielded better results.

Prompt Engineering

Each prompt (system instruction) for each agent has undergone iterative refinement, starting with a simple version in the web UI experiment and progressing throughout the development of the LLM-Assistant. The techniques used, such as few-shot prompting and structured output instructions, are inspired by the research conducted. As the system uses different agents for different tasks, each system instruction can be updated without interfering with other tasks or agents. The main lesson here was trial and error: even a small change or the order of sentences in a system instruction made a difference. An important finding was that the agents in this LLM-Assistant rely heavily on exemplars. As soon as the exemplars were removed, the agents performed notably worse, making more errors, mainly by failing to follow the Akenza scripting rules or generating malformed topic schemas.

Command-Line Interface

The LLM-Assistant can be used through a command-line interface (CLI). The Python Click library was selected for this because of its popularity and ease of use (Pallets Projects, 2025).

Feedback and Additional Creation Mode

A README file was provided with instructions on how to use the CLI. After sharing the LLM-Assistant with other developers, the initial feedback indicated that it would be helpful to have the possibility to create topic schemas and the `meta.json` file from an existing decoder script, rather than generating every artifact from scratch. This need was primarily due to the tedious nature of manually creating topic schemas. When a device manufacturer provides a generic sample decoder script, at times it only requires minor adjustments to work on the Akenza platform. An experienced developer can make these changes in just a few minutes (Gutknecht, 2025), but then still needs to create the required topic schemas and metadata file to end up with a complete device type. To address this use case, an additional mode was added to the LLM-Assistant. This mode changes the agent workflow: the topic and meta agents in this mode are instructed to create the topic schemas and metadata exclusively based on a provided, Akenza-compatible decoder script. Due to the modular design of using agents for different tasks, this was relatively easy to implement, as it only required minor adjustments to the system instructions and a change in the agent invocation order. As a result, the

LLM-Assistant now supports two modes: one that generates all device type artifacts from scratch, and another that generates only metadata and topic schemas from an existing, working decoder script.

The only artifact that is not generated, but is still needed for a complete Akenza device type, is the `uplink.spec.js` unit test. This was done deliberately to ensure extra caution when handling LLM-generated code and to make sure no errors or hallucinations end up in the Akenza device type library.

Demo

This section showcases the CLI of the LLM-Assistant. Running the CLI opens the startup menu with the following options.



Figure 5.2: CLI Startup Menu

C - Create This opens the creation menu, where the creation mode can be selected.

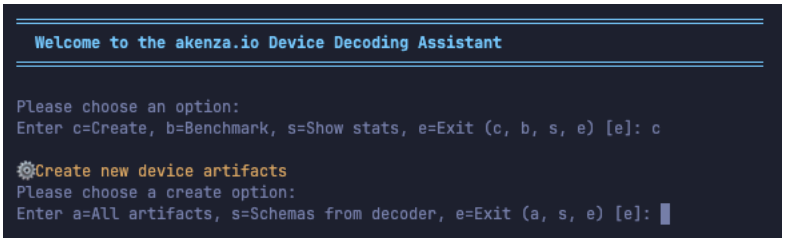


Figure 5.3: CLI Create

Create: A - All This is the mode that creates all the device type artifacts from scratch. The user is prompted to input the device name, manufacturer, and decoding logic folder. Additionally, the user is prompted to select an output folder where the generated artifacts will be saved. The agent workflow starts as soon as all required information is entered, with frequent status updates on which agent is currently running or which file has been saved.

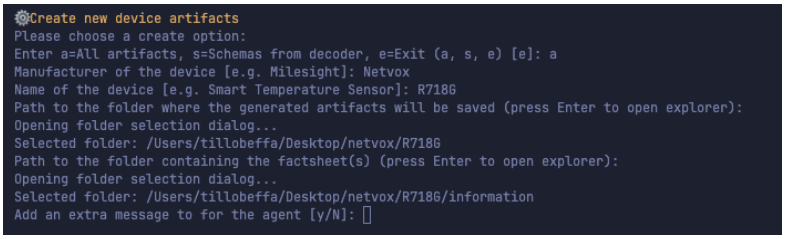


Figure 5.4: CLI Create Input

Create: S - Schemas This is the mode that creates just the topic schemas and the `meta.json` from an existing Akenza decoder script. The user is prompted to enter the device name, manufacturer, and the path to the existing decoder script. The CLI behaves the same as in the "Create all" mode.

B - Benchmark This launches the evaluation benchmarking of the LLM-Assistant. The process of benchmarking is explained in Section 5.2. To run the benchmark, only the benchmarking directory needs to be selected, which contains the benchmarking environment and a device test set.

S - Stats This outputs a results preview of previous benchmarking runs by reading a `.csv` file containing benchmarking results.

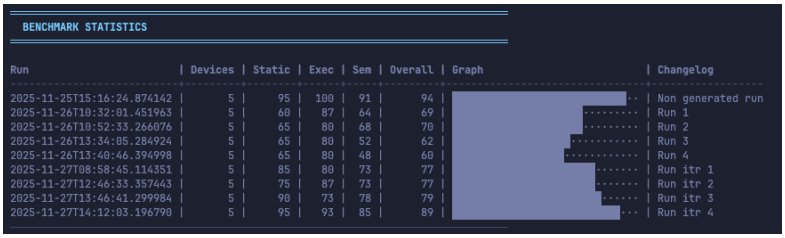


Figure 5.5: CLI Stats

5.2 Custom Evaluation Benchmark

This section describes the creation of the evaluation benchmark. The goal of this benchmark is to evaluate the LLM-Assistant and inform decisions for its refinement. It should help to quickly determine whether changes to the assistant, such as adjusting a prompt, adding a new agent, or changing the model, improve or worsen performance. As mentioned in the previous section, the initial development of the LLM-Assistant relied entirely on the manual evaluation of generated artifacts. Manual evaluation is a time-intensive process and requires pre-existing expertise in device type development. Furthermore, manual evaluation is not objective. For these reasons, an automated evaluation benchmark was developed.

5.2.1 Technology Stack

The benchmark was developed iteratively, similar to the assistant system, starting with manual evaluation and evolving until it achieved its final goal. The benchmark's main logic is implemented in JavaScript. This decision was made deliberately because the artifacts generated by the LLM-Assistant are JSON and JavaScript code. Evaluating JavaScript code and JSON with JavaScript is more straightforward than it would have been with Python. Several libraries were used, with VM2 for Node.js being the most important. VM2 allows for executing code in a sandboxed environment and easily mocking functions, making it a suitable solution for running and evaluating the generated decoder scripts and capturing their outputs (Eight04, 2023). Because the LLM-Assistant uses Python, the JavaScript code is called from within Python via subprocesses that run in a Node.js environment. Communication between the different programs happens through standard input (`stdin`) and standard output (`stdout`).

5.2.2 Benchmark Experiments

Because each device type published on the Akenza platform in the device type library includes unit tests that validate the output of a decoder script based on a sample payload, the first approach to an automated evaluation was to reuse these unit tests (Akenza, 2025a). The approach was implemented as follows: five devices that had already been created and published to the Akenza device type library were selected. The device decoding logic for each was manually researched. Finally, the LLM-Assistant was used to regenerate the topic schemas, metadata, and the decoder script, after which the existing `uplink.spec.js` unit tests were run against the newly generated artifacts.

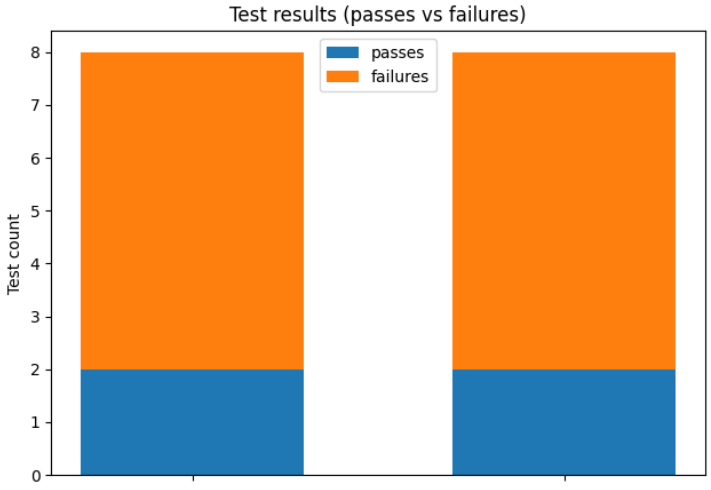


Figure 5.6: Unit Test Results
Two runs with five devices. Three devices had two unit tests while the remaining two had one. Two out of eight unit tests passed for both runs.

This approach worked and yielded the results visible in Figure 5.6. However, it also revealed a major flaw. Nearly all of the unit tests failed, yet upon manual inspection, the decoder scripts appeared better than the low pass rate (25%) suggested. The problem was that the unit tests were written very narrowly, meaning that only an exact match would be valid. Because the decoded output of the decoder script can have multiple valid forms, the unit test pass/fail rate was not a meaningful metric.

Multiple Valid Forms

The following example illustrates the different valid forms a decoded payload can take. The Netvox R712 is a wireless outdoor temperature and humidity sensor (Netvox, 2025). A sample hexadecimal payload for this device is shown in Figure 2.1, and its corresponding decoded output using an Akenza device type is shown in Figure 2.2.

In the payload `01010124067 01A9E000000`, the bytes `0670` represent a temperature of 16.48°C, and `1A9E` represents a humidity level of 68.14%. The ninth byte, `00`, indicates the status of various threshold alarms, such as high or low temperature. Although the data itself is fixed, there are several different, correct ways to decode and structure this information.

Variations in Property Names, Formats, and Topics

- **Property Names:** The field representing the low temperature alarm could have various valid names, such as `lowTemperatureAlarm`, `tempAlarmLow`, or `alarm`.

- **Data Formats:** The value for a triggered alarm can also be represented in various formats. It could be a boolean (`true` or `false`), an integer representing a bitmask, or a descriptive string like `"high_temp"`. It could also have a completely different format where, instead of a separate property per alarm, all active alarms are grouped into a single property and represented by an array of active alarms, such as `["lowTemperature", "highHumidity"]`.

- **Topic Categorization:** The decoded data can also be categorized into different topics. Akenza does not have a fixed set of possible topics. The temperature and humidity data could be placed in a `default` or `environment` topic, while the alarms could be placed in an `alarm` or `alert` topic. The Akenza device type for this device groups all data together under the `default` topic.

Creating unit tests that would correctly test for all these different variations is very difficult and time-intensive. This is especially true when considering that it would be beneficial to periodically change the devices in the test set used for the benchmark to avoid overfitting the LLM-Assistant. In addition, these unit tests

fail to validate the form of the device type artifacts, as they focus only on the final output.

5.2.3 Evaluation Benchmark Implementation

This section explains how the final evaluation benchmark works and how it was implemented. The previous experiment identified that relying on simple unit tests to evaluate the LLM-Assistant's performance was insufficient. For this reason, the focus shifted toward validating all requirements regarding the form of the different artifacts. Additionally, other methods were used, such as LLM-as-a-judge, to validate semantic correctness.

Validating the form of the different artifacts first required identifying all of the requirements an artifact must meet. These findings are listed in Appendix A.6.

The identified requirements are grouped into four categories: Syntactic Correctness, Execution Correctness, Semantic Correctness, and Code Similarity. Each of these categories contains multiple metrics, and each metric is assigned a corresponding weight. This makes it possible to prioritize certain metrics over others. For example, it is more important that the overall syntax of the decoder script is correct than it is for a single schema property to have a non-conforming name. All the metrics are added together to create a score between 0 and 100. The different categories of the evaluation benchmark are executed sequentially in the order listed below.

Syntactic Correctness

Raw Syntax This section evaluates whether the decoder script and JSON files have valid syntax. The syntax check for the decoder script is performed by converting the code into an Abstract Syntax Tree (AST). This process fails automatically if the syntax is incorrect. For JSON validation, Another JSON Schema Validator (AJV) is used to verify the structure (Ajv, 2025). These syntax checks result in the first three metrics:

- Valid `uplink.js` syntax
- Valid `topic.schema.json` syntax (each of the topic schemas is tested)
- Valid `meta.json` syntax

Rule Adherence The Syntactic Correctness category also includes rule adherence checks. For these checks, the decoder code and JSON files are examined in detail.

- **Decoder Script:** This check can only begin if the script's syntax is valid, as it relies on traversing the entire Abstract Syntax Tree. For each rule,

the evaluation script examines the corresponding AST nodes to determine whether they satisfy the requirements. For example, to verify whether the `emit()` function is called with the correct signature, the AST is scanned until an `emit` call expression is found. The script then analyzes the type of call and verifies that all arguments passed to the function are valid. If an issue is found, it is appended to a device issue list.

- **JSON Artifacts:** Both JSON artifacts, the topic schemas and the metadata, are validated against additional JSON schemas using AJV. This check also ensures that all referenced properties inside the topic schemas are valid. It achieves this by fetching all relevant remote references and validating them through AJV.

Based on these rule adherence checks, a total of nine metrics are calculated for the syntactic correctness category. Each is weighted by its relative importance and added to the final score.

Execution Correctness

This stage is initiated only after the decoder script and topic schemas have been verified to have valid syntax. It checks that the decoded data emitted by the decoder script using a sample payload, conforms to its corresponding generated topic schema. To achieve this, the decoder script is executed within a Node.js VM2 sandbox.

Real payload samples must be passed into the `consume(event)` function. For this purpose, a `ground_truth.json` file was created for each device in the benchmark test set. This ground-truth JSON is explained in Section 5.3.1 and includes sample payloads for a device along with the desired decoded data. These payloads are passed to the `consume(event)` function, the script is executed, and the `emit()` function is mocked. Each output from the `emit()` function is appended to an array.

The contents of this array are then validated against the generated topic schemas using AJV. The score is calculated as a success rate; for example, if four out of five emits conform to the schema, it results in a score of 0.8. Passing this check confirms that the decoded data structure is valid according to the topic schema. However, it does not determine if the actual values emitted are correct.

Semantic Correctness

The most challenging part of the evaluation benchmark is verifying that the decoded data emitted by a decoder script is semantically correct. As previously mentioned, it is very difficult to use standard unit tests for this purpose. A possible solution is using Large Language Models to evaluate the decoded payload against a correct solution from the `ground_truth.json`.

The approach works as follows: an LLM-as-a-judge agent was created with the instruction to check the semantic correctness of the decoded payloads. The system instruction for this agent can be found in Appendix A.5. The agent is provided with the captured decoded values from the mocked `emit()` function, a correct solution from the `ground_truth.json`, a reference decoder script from the Akenza device type library, and the generated decoder script. The agent not only checks whether the values appear to be correct but also identifies if any values are missing. Finally, the agent is instructed to score the script output on a scale (between 0 and 1).

Using an LLM to check semantic correctness introduces a new set of challenges. The first is reliability; experiments showed that the judge agent tends to produce slightly different output scores when given the same input across different runs. However, it was observed that the general direction and trend of the scores remained consistent. An additional problem is hallucinations. Hallucinations are not only a risk during the generation of artifacts by the LLM-Assistant but can also occur during the LLM-as-a-judge evaluation, potentially leading to misjudgment. To address this, the semantic correctness scores from the LLM were tested multiple times against manual evaluations, and a strong correlation was found. For this reason, this metric is considered to contribute meaningfully to the overall benchmark score. Consequently, this score is weighted and included in the final evaluation.

Code Similarity

As explained in Chapter 2, similarity metrics are often used to evaluate LLM outputs. For this reason, the evaluation benchmark includes CodeBLEU metrics. CodeBLEU compares a reference decoder script from the device type library with its LLM-generated counterpart to calculate a similarity score. The benchmark uses the CodeBLEU implementation from Microsoft's open-source CodeXGLUE (Microsoft, 2020). CodeBLEU calculates the following four metrics, each of which has a weight of 0.25 for the overall CodeBLEU score:

- N-Gram Match
- Weighted N-Gram Match
- Syntactic AST Match
- Semantic Data-Flow Match

During experiments using CodeBLEU to evaluate the quality of the generated decoder scripts, the scores were consistently low, even for scripts that were fully working and decoded correctly. For this reason, the total weight of the CodeBLEU score was reduced, contributing only 0.1 to the overall benchmark result.

The weights for all metrics were set manually after evaluating their relative importance through experimentation.

Overview of Metrics and Weights

The following table provides a complete overview of the weights assigned to each category and metric.

Category (Weight)	**Sub-Category (Weight)**	**Metric**	**Relative Weight**
Syntactic (0.25)	Script (0.5)	Syntax Valid	0.2
		Consume()	0.2
		Emit()	0.2
		Self-Contained	0.1
		Topic Conform	0.1
		DataKey Conform	0.1
		Console Conform	0.1
	Metadata (0.25)	Syntax Valid	1.0
	Topic Schemas (0.25)	Syntax Valid	1.0
Execution (0.25)	Payload Conformance	Success Rate	1.0
Semantic & Similarity (0.5)	Similarity	CodeBLEU	0.2
	Logic Accuracy	LLM-as-a-Judge	0.8

Table 5.1: Hierarchical Overview of Benchmark Metrics and Weights

Demo

The evaluation benchmark can be executed on a single device or on a set of devices (a test set). The following screenshots show a benchmark run for a test set consisting of five devices, focusing on the results for the Vega Vegapulse Air 41 device, which achieved a near-perfect score.



Figure 5.7: Benchmark initialization

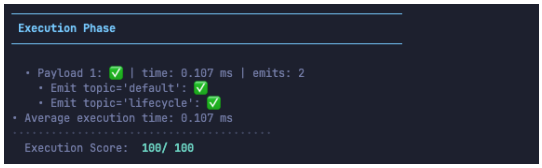


Figure 5.8: Execution and contract testing

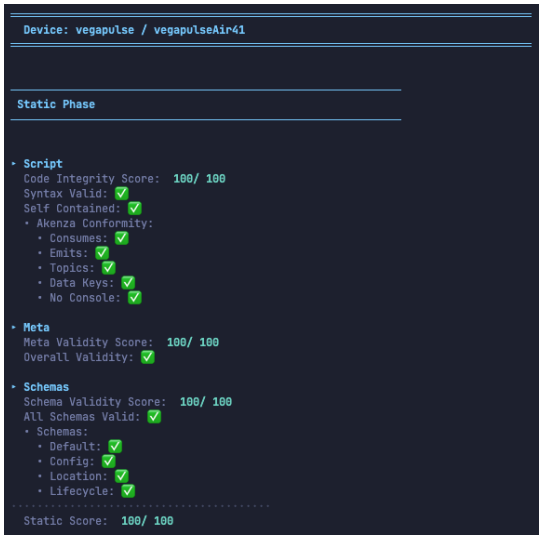


Figure 5.9: Syntax validation phase



Figure 5.10: Semantic scoring phase

5.2.4 Issue List

If checks fail during the evaluation benchmark, an issue is produced and appended to a device issue list. This list helps to identify systematic failures in the LLM-Assistant. As an example, the most common issues found during four runs of five devices in the first refinement cycle are listed in Appendix A.8.

5.2.5 Benchmark Meta-Evaluation

As identified in Section 3.3.1, an evaluation benchmark that outputs a score is only useful if this score correlates with the actual quality of the evaluated artifacts. To ensure this, a meta-evaluation was conducted. The meta-evaluation consisted of two experiments.

Experiment 1: Correlation First, the LLM-Assistant was configured to use two different LLMs exclusively for each agent. Gemini 2.5 Pro and Gemini 2.5 Flash were selected. The hypothesis was that the Flash model would achieve

lower scores than the Pro model, and that the evaluation benchmark scores would correlate with manual evaluation scores. The LLM selection was based on results from popular public LLM capability benchmarks (LLM Stats, 2025) showing that Gemini 2.5 Flash is less capable than Gemini 2.5 Pro. The devices used for this experiment are listed in Appendix A.7.

The results, shown in the following table, confirm the hypothesis. They show a clear positive trend. The manual evaluation followed the same rubric as the LLM-as-a-judge, detailed in Appendix A.5.

Model	**Benchmark Score (%)**	**Manual Evaluation Score (%)**
Gemini 2.5 Pro	60	75
Gemini 2.5 Flash	24	50

Table 5.2: Meta-Evaluation Results: Pro vs. Flash Model

Experiment 2: Validation In the second experiment, five devices were selected from the Akenza device type library, along with their manually created and published artifacts. The hypothesis was that these complete, human-authored artifacts would yield a perfect score in the evaluation benchmark.

The benchmark returned a score of 94%. This score was significantly higher than any achieved by a test set with LLM-generated artifacts. The issues list showed that the few points deducted were for minor rule violations where property key names contained underscores, which is technically not allowed.

These two experiments provided sufficient evidence to conclude that the scores produced by the evaluation benchmark meaningfully reflect the performance of the LLM-Assistant and the quality of the generated artifacts.

5.3 Iterative Refinement Cycle

Both the LLM-Assistant and the evaluation benchmark were developed independently, following an iterative process. Since the evaluation benchmark was created after the LLM-Assistant, it was not available to guide the assistant's initial development. This section explains how the evaluation benchmark was subsequently used to improve the LLM-Assistant and create a data-driven refinement cycle.

5.3.1 Test Set

To get a better indication of the LLM-Assistant performance a test set including five devices with varying complexity was created. This set can be reused for the evaluation benchmark to reliably indicate whether modifications to the assistant affect its performance. The devices in this test set can be found in Appendix A.7.

Each device in this list is accompanied by a `ground_truth.json` file, which contains sample payloads and a variant of the expected decoded payload. The expected decoded payload is separated into topics and properties. Each property includes a list of possible valid names, which are used by the LLM-as-a-judge for more accurate semantic validation. An example of a `ground_truth.json` file can be found in Appendix A.3.

Each device in the test set is also accompanied by a `.txt` or `.pdf` file containing device decoding information, which is used as initial input for the LLM-Assistant's generation step.

5.3.2 Results of the Refinement Cycle

The refinement cycle followed the iterative process described in Section 5.3.

Initial Benchmark Results The evaluation benchmark was run four separate times on the test set. Because the LLM-Assistant does not generate identical artifacts for the same input, multiple runs were necessary to establish an average baseline score. The initial results are shown in the following table.

Run No.	Syntactic Score (%)	Execution Score (%)	Semantic & Similarity Score (%)	Overall Score (%)
1	60	87	64	69
2	65	80	48	60
3	65	80	68	70
4	65	80	52	62
Average Overall Score				**65.25**

Table 5.3: Benchmark Scores Before Refinement

Refinement Process Based on these four runs, the issues identified by the benchmark (see Appendix A.8) and through manual review (see Appendix A.9) were systematically addressed. Each issue was fixed on a per-agent basis, with a rerun of the evaluation benchmark on a single device to verify that the fix was effective. The primary adjustments were:

- Refining the system instruction for each agent.
- Upgrading the models for specific agents: The models for the topic and decoder agent were both updated from Gemini 2.5 Pro → Gemini 3.0 Preview.

Refinement Benchmark Results After all adjustments were implemented, the evaluation benchmark was run again four times with the same test set. The resulting scores are visible in the following table.

Run No.	Syntactic Score (%)	Execution Score (%)	Semantic & Similarity Score (%)	Overall Score (%)
1	85	80	73	77
2	75	87	73	77
3	95	93	85	89
4	90	73	78	79
Average Overall Score				**80.50**

Table 5.4: Benchmark Scores After Refinement

Results The implemented changes raised the overall benchmark score from 65.25% to 80.5%. This represents a relative increase of 23.4% and an absolute increase of 15.25 percentage points. Upon manual inspection of the generated artifacts, this quantitative refinement was confirmed by a noticeable increase in quality.

Token Usage and Cost

This section analyzes the average token usage and associated costs for the generation of all artifacts in a full evaluation benchmark run (with five devices) both before and after the refinement process.

The model costs used for these calculations are listed in Table 5.5 and are based on non-cached pricing, as the amount of cached tokens was not captured during the experiments.

Model	Input Cost (per 1M tokens)	Output Cost (per 1M tokens)
Gemini 2.5 Flash	\$0.30	\$2.50
Gemini 2.5 Pro	\$1.25	\$10.00
Gemini 3.0 Pro Preview	\$2.00	\$12.00

Table 5.5: Gemini Model Pricing as of 22.12.2025 (Google, 2025b)

Before the adjustments, an average run consumed approximately 350,000 input tokens and 40,000 output tokens, resulting in a total cost of about \$0.6.

After the LLM-Assistant was adjusted, the input token usage was reduced to 300,000 tokens, mainly due to making the agent system instructions more compact. The output token count remained the same. However, as the model of the decoder and topic agent was changed to the newer Gemini 3.0 Pro Preview, the total cost for a full run increased to $0.75.

5.4 Summary of Implementation

In conclusion, this implementation showcases a robust methodology for building and refining an LLM-enabled assistant in a domain-specific field. While the LLM-Assistant was developed as a proof-of-concept, the significant focus on creating a rigid evaluation framework is a critical takeaway.

It is important to mention that such a significant focus on an automated and objective evaluation may not be justifiable or the best fit for every experimental project. A manual evaluation might offer a better return on investment in the early phases when the fundamental utility of an LLM-Assistant is still unproven. However, once an LLM-Assistant demonstrates value, the methodology presented here provides a scalable and objective pathway to achieving high levels of quality and reliability.

Chapter 6

Validation and Evaluation

To answer the thesis's research question, a comparative case study was conducted to validate the LLM-Assistant and quantify its practical value in terms of time saved.

6.1 Case Study Setup

The study was separated into two phases:

- **Manual Creation:** A developer creates a complete device type from scratch by hand.
- **LLM-Assisted Creation:** The same task is performed by the same developer with the help of the LLM-Assistant.

Two possible options were considered for selecting the devices for which to create the device type in the case study:

- **Option 1:** Using the same device for both phases. This approach is somewhat unfair as the second attempt at creation would be easier and probably faster, given the prior experience gained during the first creation.
- **Option 2:** Using different devices with similar complexity and from the same manufacturer. While also not perfectly balanced due to possible differences in device complexity, this second option seems more reliable for quantifying time savings.

The second option was chosen, and to further mitigate bias, an additional developer was tasked with conducting initial research and find two devices from the same manufacturer with comparable levels of complexity.

The following two devices were selected:

- Dragino DDS20-LB (Dragino, 2025b)
- Dragino DDS04-LB (Dragino, 2025a)

For both phases of creation, a time-tracking table was used to log time spent on the different stages of device type creation. Additional notes were taken to detail the specific actions performed in each stage. A creation phase was considered complete once the created device type passed peer review and the pull request was merged into the Akenza device type library.

The case study was conducted by a developer with 4 months of experience in device type creation who had already used the LLM-Assistant.

Note: As this comparative case study had only one participant and was executed once, the findings cannot be generalized but rather indicate the potential for time savings.

6.1.1 Results

The following tables showcase the time spent on each stage for both the manual and LLM-assisted creation processes. Both finished device types can be found in the Akenza device type library on GitHub[4]. The notes for each stage can be found in Appendix A.10.

Stage	**Manual Time**	**LLM-Assisted Time**	**Time Saved**
Setup	2min	3min	-1min
Research	8min	8min	0min
Decoder Script	86min	20min	66min
Topic Schemas	36min	8min	28min
Testing	34min	23min	11min
Metadata	4min	1min	3min
PR	11min	2min	9min
PR Comments Fix	21min	35min	-14min
Total Time	**202 min**	**100 min**	**102 min**

Table 6.1: Time Tracking Comparison: Manual vs. LLM-Assisted Creation

An approximately 50% (102 min) time reduction was observed when comparing the manual versus LLM-Assisted creation. The most time was saved during the

[4]Akenza Dragino device types: `https://github.com/akenza-io/device-type-library/tree/main/types/dragino`

artifact creation stages. The additional time spent resolving PR comments was largely due to date mocking requirements in tests and was not a result of poor code quality.

6.2 Successful Deployments of the LLM-Assistant

Following the development phase described in Section 5.1, the LLM-Assistant was made available to Akenza developers. It has already been successfully used in a customer project, where it contributed to creating six device types from scratch that were published in the device type library within a single day.

For each of these device types, the assistant was used to generate all the necessary artifacts. These artifacts then served as a starting point and only needed minor adjustments to be finalized.

The additional mode described in Section 5.1.3, where the LLM-Assistant creates topic schemas and metadata based on a working Akenza decoder script, has also been successfully used in production (Gutknecht, 2025).

6.3 Refinement Cycle Validation

It is important to address two points regarding the results of the refinement cycle from Section 5.3.

First, the relative increase of 23.4% should be contextualized. The LLM-Assistant was intentionally kept in its initial, unrefined state while the evaluation benchmark was being developed. This was a deliberate decision to more clearly demonstrate the potential for data-driven refinement once the evaluation benchmark was implemented. Thus, such an increase can probably not be expected after every refinement cycle.

Second, the repeated use of a narrow test set (five devices) introduces a risk of overfitting. The assistant may become highly optimized for this specific set of devices and fail to generalize well to others.

Therefore, this experiment should be viewed as a successful demonstration of the refinement cycle's efficacy, rather than a guarantee of a specific performance increase across all scenarios.

6.4 Return on Investment Analysis

To evaluate the practical value of the LLM-Assistant, a simplified Return on Investment (ROI) analysis was conducted. This analysis connects the development

effort to projected time savings and aims to provide an estimate of the break-even point for the project.

6.4.1 Assumptions and Limitations

It is important to note that this analysis is a high-level projection and relies on several key assumptions:

- The 50% time saving observed in the single-participant case study is assumed to be representative of the average saving across all future projects.
- The hourly cost of developing this project is considered equivalent to the hourly cost of creating a device type.
- Future costs related to maintenance, updates, and bug fixes for the LLM-Assistant are not included in this calculation.
- The operational cost of the LLM-Assistant (token usage) is considered negligible.

Given these assumptions, this analysis should be interpreted as an optimistic estimate of the potential return.

6.4.2 Development Investment

The time spent on the development of this project is broken down as follows, excluding time spent on research and the comparative case study:

- Initial LLM-Assistant development: 72 hours
- Evaluation system and benchmark development: 160 hours
- Dataset curation and iterative refinement cycles: 16 hours

This results in a total development time of approximately 248 hours.

6.4.3 Projected Savings and ROI

Akenza creates approximately 50 new device types per year, with each taking an average of three hours to develop (Gutknecht, 2025). Based on the 50% time saving, using the LLM-Assistant would save approximately 75 hours per year. If annual creation increased to 100 new device types, the annual savings would double to 150 hours.

The initial investment would be recouped in approximately 3.3 years for 50 device types per year, or 1.65 years for 100 device types per year.

It is also important to note that the expertise acquired through this project can be applied to other similar projects. This knowledge transfer increases the efficiency and viability of future LLM-enabled projects at Akenza.

6.5 Answering the Research Question

> *How, and to what extent, can an LLM-based system be used to assist and accelerate the development process in a domain-specific area such as IoT payload decoders?*

The implementation of the LLM-Assistant and the results of the comparative case study indicate that an LLM-based system can successfully accelerate development in the domain of Akenza IoT payload decoders by approximately 50%. The "how" was addressed through an agent-based architecture in which agents with distinct system instructions and tasks collaborate sequentially to generate a complete set of device type artifacts. These artifacts serve as a helpful starting point for further adjustment and refinement. However, the journey to this result required a significant upfront investment, particularly in creating an automated, objective evaluation benchmark that enabled data-driven iterative refinement.

6.6 Summary

This thesis resulted in the development of an agentic LLM-Assistant system capable of generating all essential device type artifacts, as well as the creation of an evaluation benchmark to meaningfully assess the assistant's performance and rapidly determine the impact of changes to its models, prompts, and system design. This benchmark enabled an iterative, data-driven refinement process, resulting in a relative increase of 23.4% in the benchmark score after a single cycle. A case study comparing manual versus LLM-assisted development time indicated approximately 50% time savings. The LLM-Assistant, while still a prototype, has already been successfully used in production by Akenza developers and in a customer project. All objectives outlined in the problem definition were successfully achieved within the stated project timeline (Figure 4.1).

Chapter 7

Outlook

Further development of the LLM-Assistant and its integration into Akenza's existing LLM product are planned. The design and technology choices made during prototyping are expected to enable a straightforward integration process.

7.1 Areas of Improvement

Two main areas for immediate further improvements were identified.

7.1.1 Human-in-the-Loop

The current LLM-Assistant operates as an autonomous workflow, initiated by a user and running from start to finish without further interruption. While convenient, this process can be sensitive to errors that propagate through subsequent steps.

A well-designed human-in-the-loop approach could help address such errors. The approach could look as follows: The workflow would pause after each agent's generation step, requiring user validation before proceeding. If the generated content is not satisfactory, the user could either make manual adjustments directly or provide feedback to the agent for regeneration.

7.1.2 Context Management

Another area for improvement is context management between agents. The current implementation takes a straightforward approach by feeding the inputs and outputs of previous agents into subsequent ones. This method was simple to implement and relatively effective, particularly because the models used support large context windows. However, a potential drawback is that the agent's context can quickly become bloated, especially when initial user inputs are provided in `.txt`

and `.pdf` files. These files may contain additional information irrelevant to device type generation, potentially negatively affecting the quality of the generated artifacts. A potential solution is to introduce filters to distill relevant information from the initial user input and between agents.

7.2 Conclusion

This thesis demonstrated that LLMs can streamline coding-related processes in domain-specific areas. It also showed that an iterative development process with a strong focus on evaluation is effective. The resulting LLM-Assistant and evaluation logic indicated approximately 50% time savings and serve as a foundation for the next stage of development. As the refinement cycle showed that model updates and system instruction changes directly increase code generation quality, it is to be expected that future, newer models will further contribute to an even better LLM-Assistant, thus potentially saving even more time in device type creation. The goal is to evolve this LLM-Assistant into a product that will empower both Akenza developers and platform clients, lowering the entry barrier and making the complex, time-consuming task of IoT device integration more efficient.

Appendix A

Supporting Materials

A.1 Interview Transcripts

A.1.1 Interview: IoT Decoder Creation Process

Date: December 2, 2025
Interviewer: Tillo Beffa
Interviewee: Jonas Gutknecht, IoT Solution Engineer at Akenza
Method: Google Meets interview with automatic transcription

Part 1: The Creation Process & Workflows

TILLO BEFFA: How does the process of creating a device type normally look for you, from start to finish?

JONAS GUTKNECHT: It usually starts when a client requests a device type. There are two main scenarios here:

1. Personal Use: If the device doesn't need to be in the Device Type Library and is just for a specific project, I will quickly write one. For example, if it's a Milesight device or something similar with an already prepared decoder, this process is very fast, usually around 10 minutes.
2. Device Type Library: If the device needs to be included in the official library, the process is longer because it requires creating schemas, automated tests, and proper documentation.

Let's stick with the Milesight example for the library workflow. The steps are usually:

- Copy & Rewrite: I take the existing decoder and rewrite it to conform to our language styling.

- Restructure: I rewrite all data point names and add topics for structure.
- Schemas: I create schemas based on the uplink or the script I just created.
- Testing: I add automated testing using the payloads provided by the manufacturer in their documentation.
- Metadata: Finally, I add the meta file.

Depending on the complexity of the device, this full process takes anywhere from 1 hour to roughly 12 hours.

Part 2: Research & Missing Documentation

TILLO BEFFA: How does the process look if you don't have a decoder script provided by the manufacturer? What kind of research do you do before starting?

JONAS GUTKNECHT: If I lack information, I usually start by Googling and checking various GitHub repositories to see if something already exists.

If nothing exists, I get the payload documentation directly from the manufacturer and write the decoder manually. We have some helper functions that make this easier, but every manufacturer has its own quirks, so it takes time.

I mentioned earlier that some devices can take up to 12 hours. This is usually the case for bridges or devices that send variable data points. For example, if you connect many smaller devices to one bridge, it becomes complicated because you often have to choose channels or configure the device with specific commands.

Part 3: Testing & Payloads

TILLO BEFFA: You mentioned the requirement to create unit tests with payloads. Is it ever the case that you don't have any payloads to test for? If so, do you even start creating the device type?

JONAS GUTKNECHT: There are some exceptions where devices do not have tests, but that is rare. If the manufacturer does not provide a payload, I usually set up the physical device myself, run it without the decoder, and extract the payloads through that process.

However, this is mostly a sanity check. Because I don't have an official example, I can't be 100% sure the data is correct; I have to compare with similar sensors in the area that the decoded data is correct.

Part 4: Time Estimation

TILLO BEFFA: You gave a wide time range earlier. What is the typical time required? How do you plan for how much time a device type will take?

JONAS GUTKNECHT: Most devices take about two to three hours.

It is important to note that the most time-consuming task is not scripting the decoder itself. The biggest tasks are actually documenting the data points and creating the automated testing.

Part 5: AI Assistance in the Workflow

TILLO BEFFA: Regarding a potential device assistant or LLM (Large Language Model), where do you see such a tool being most helpful? Would it be in creating the scripts or the documentation? Conversely, where might it struggle?

JONAS GUTKNECHT:

- Documentation: I have already used AI successfully for documenting. It saves time; even if the output isn't totally correct, it provides a skeleton that I can work with.
- Scripting: For a device that already has a decoder, rewriting it doesn't take me much time personally, though it might be helpful for an inexperienced user. However, the user would still need to verify the output.
- Limitations: I think AI runs into problems when dealing with purely text-based instructions, like a PDF or a text file. It often struggles to create accurate code based solely on unstructured text descriptions.

Part 6: Experience

TILLO BEFFA: Just to get a bit more context, how much experience do you have creating device types?

JONAS GUTKNECHT: I have five-plus years of experience.

TILLO BEFFA: Okay that is all, thank you for the interview.

JONAS GUTKNECHT: No worries.

A.1.2 Interview: The Evolution of Genio and Agentic Systems

Date: December 3, 2025
Interviewer: Tillo Beffa
Interviewee: Janick Rüegger, Senior Software Engineer at Akenza
Method: Google Meets interview with automatic transcription

Part 1: Background and Motivation

TILLO BEFFA: How did Genio start? Could you tell me about the background of the project?

JANICK RÜEGGER: Sure. When Akenza initially started, the main hypothesis was that we needed to simplify the setup of IoT so that many organizations could profit from those capabilities. However, we quickly realized that you need to be able to get really good data and insights out of your IoT setup, otherwise, it is basically worthless.

The next hypothesis was that if we onboarded big corporations, they would bring their own data science teams to analyze the data. In reality, most companies, even the bigger ones, struggle with this.

We started investing in out-of-the-box analytics capabilities, but building those features took a lot of time and investment. Genio is really an effort to allow non-technical users to use their IoT data out of the box. It isn't just for repetitive features or use cases, but also for specific, one-off instances where they need insights quickly. Previously, building dashboards took non-technical users multiple hours. Genio opens up opportunities by reducing the time-to-insight from hours to seconds. This makes the data setup at Akenza much more valuable because the initial barrier to entry is so much lower compared to using standard dashboards.

Part 2: Timeline and Architectural Evolution

TILLO BEFFA: When did the planning for Genio start? What was the timeline?

JANICK RÜEGGER: Genio started roughly in March of last year, so we have been working on it for about a year and a half. It is important to highlight that the idea of what Genio is, its capabilities, and what we tried to achieve matured over time.

At the very beginning, the scope was much narrower. Our initial goal for the first two months was to retrieve really narrow data, and the

prompts needed to be extremely detailed—you almost needed to provide specific IDs. We have switched architectures at least twice since then.

1. LLM Call: At the start, it was basically just a direct LLM call.
2. Simple RAG: We then moved to a simple RAG (Retrieval-Augmented Generation) architecture with one retrieval and one generation step.
3. Router RAG: Next, we moved to a router RAG architecture where we provided multiple business domains that were not connected, which you had to query individually.
4. Static Workflows: We then moved to static workflows, which you can think of as binary decision trees, where an LLM decided at each step if it needed business-bound data.
5. Agentic System: Nowadays, we are working on an Agentic system. It is dynamic, multi-turn, and has memory. It is much more complex, but it allows us to combine different business domains that we previously could not connect.

Part 3: Technology Stack and Frameworks

TILLO BEFFA: How has the technology stack evolved, and what does the current architecture look like in terms of frameworks and programming languages?

JANICK RÜEGGER: The language used has been Python from the start. While there are interesting projects in TypeScript, Go, or Java, Python remains the main ecosystem for data science and the generative AI advances we see today (such as LangChain, LangGraph, Pydantic AI, etc.).

Regarding the libraries:

- LangChain: We started with LangChain because it was published shortly after our first investigations. We built a good Proof of Concept (PoC) with it, and our first drafts relied on it.
- The Shift to Agents: As we moved to an Agentic system, we needed more support for planning and orchestration, which LangChain didn't fully provide out of the box. LangGraph was released to cover that aspect.
- Selection Process: We researched various technologies and concluded we should do a PoC with LangGraph and Google ADK. Back then, the Google ADKs ecosystem seemed to be the better

fit because it supported *a priori* planning. Since we value latency highly, we chose the Google ADK as our main framework.

Regarding data pipelines and storage:

- Databases: We started using only data from MySQL. We have since onboarded ClickHouse, which proved to be a real game-changer for us in terms of latency. We are focusing heavily on data engineering and modeling there.
- APIs: We use different REST APIs in our agents and tools. We are also including a new API leveraging GraphQL, Neo4j, and ClickHouse in the background.

A.2 Python Project Configuration

```
[project]
name = "device-decoding-assistant"
version = "0.1.0"
description = "A tool for extracting device information from
   PDFs and generating decoding scripts using Google ADK"
readme = "README.md"
requires-python = ">=3.13.5"
dependencies = [
    "pdfplumber>=0.11.7",
    "python-dotenv>=1.1.1",
    "openai>=1.98.0",
    "pydantic>=2.11.7",
    "pydantic-settings>=2.7.0",
    "opik>=1.8.18",
    "jsbeautifier>=1.15.4",
    "google-adk>=1.10.0",
    "pandas>=2.3.1",
    "matplotlib>=3.10.5",
    "click>=8.2.1",
    "tree-sitter>=0.22,<0.23",
    "tree-sitter-javascript>=0.21,<0.22",
    "codebleu",
]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.hatch.build.targets.wheel]
packages = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src"]
```

```
33
34 [dependency-groups]
35 dev = [
36     "pytest>=7.0.0",
37     "black>=23.0.0",
38     "flake8>=6.0.0",
39     "mypy>=1.0.0",
40 ]
41
42 [tool.uv.sources]
43 codebleu = { git = "https://github.com/k4black/codebleu.git",
       rev = "0.7.1" }
44
45 [tool.pyright]
46 ignore = ["reportUnknownVariableType", "
       reportMissingModuleSource" ]
```

Listing A.1: Configuration of the development environment (pyproject.toml)

A.3 Ground Truth Example

`ground_truth.json` of the DecentLab IAM device

```
1  {
2    "device_name": "IAM",
3    "device_manufacturer": "Decentlab",
4    "payload_samples": [
5      {
6        "description": "Default payload",
7        "payload": "020
       bbd007f0b926a515d48bc4e0262006981c7000093d4000b0111",
8        "payload_port": 2,
9        "expected_emits_count": 2,
10       "expected_emits": [
11         {
12           "topic": "default",
13           "data": [
14             {
15               "name": "temperature",
16               "value": [27.68],
17               "alternate_name": ["temp", "air_temperature", "
       airTemperature"],
18               "type": "number",
19               "optional": false
20             },
21             {
22               "name": "humidity",
23               "value": [36.44],
24               "alternate_name": [
25                 "hum",
26                 "relative_humidity",
```

```
                "relativeHumidity"
              ],
              "type": "number",
              "optional": false
            },
            {
              "name": "pressure",
              "value": [964.12],
              "alternate_name": ["barometer", "air_pressure", "
    airPressure"],
              "type": "number",
              "optional": false
            },
            {
              "name": "co2",
              "value": [455],
              "alternate_name": ["CO2", "carbon_dioxide"],
              "type": "number",
              "optional": false
            }
          ]
        },
        {
          "topic": "occupancy",
          "data": [
            {
              "name": "occupancy",
              "value": [2],
              "alternate_name": ["occupancy_status", "
    occupancyStatus"],
              "type": "number",
              "optional": false
            },
            {
              "name": "occupied",
              "value": [true, 1],
              "alternate_name": ["is_occupied", "isOccupied"],
              "type": ["boolean", "number"],
              "optional": false
            },
            {
              "name": "minutesSinceLastOccupied",
              "value": [0],
              "alternate_name": ["minutes_since_last_occupied"],
              "type": "number",
              "optional": false
            }
          ]
        }
      ]
    }
  ]
}
```

A.4 Initial LLM System Instructions

```
--- System Instruction ---
You are a helpful assistant specialized in IoT device
   payload decoding.

You have expertise in:
 - Decoding various device payloads from Hex (LoRaWAN) or
   raw JSON formats into human-readable data.
 - Understanding sensor data formats and protocols.
Creating and debugging decoding scripts in JavaScript.

You should:
 - Create the complete JavaScript code for a decoding
   script for the following device. You will be provided
   with a sensor factsheet and all the additional
   information you need to create the decoding script.
 - The script should be able to decode the payload of the
   device and return the data in a human-readable format.
 - The most important requirement is that you create a
   completely working and finished script that is instantly
    ready to use.
 - The decoder script needs to follow all Akenza Platform
   rules.

--- First User Prompt ---
Create the decoder script for {manufacturer name, device
   name} using the following decoding logic:
{Copy of device decoding logic}
```

Listing A.2: System instruction and first user prompt

A.5 LLM-As-A-Judge System Instructions

```
You are an expert AI assistant for IoT development. Your
   primary function is to meticulously analyze an IoT
   device's data 'Payloads', compare them against the '
   Ground_Truth' specification, and, most importantly,
   debug the provided 'Decoder_Script' to identify the root
    cause of any discrepancies.

Your goal is to provide specific, actionable feedback to
   help a developer fix their code.

## Hierarchical Evaluation Process
```

```
You must follow this specific order of analysis. The
   presence of data is more important than its structure.

1.  **Global Data Point Audit (Highest Priority)**:
    - First, for a given ground truth sample, create a
   checklist of all **non-optional** data points (`"
   optional": false`) expected across ALL its topics.
    - Scan through ALL data emitted for the corresponding
   payload, regardless of the topic it was emitted on.
    - **Your primary check is: Is every required data point
    present *somewhere* in the output?** A missing data
   point is the most severe type of error. It is ok if the
   data point is emitted under a different topic than
   expected.

2.  **Structural and Value-Based Analysis**:
    - After verifying which data points are present,
   analyze the structure and values.
    - **Topic Correctness**: Is the data point emitted
   under the correct `topic` as specified in the ground
   truth? (e.g., is `batteryVoltage` under `lifecycle`?).
   Emitting data on the wrong topic is a minor error if the
    data itself is present and correct.
    - **Name Correctness**: Does the emitted field name
   match the `name` an `alternateName` or something similar
   ?
    - **Value Correctness**: Is the emitted value correct?
        - **Numbers**: Allow for reasonable rounding
   differences.
        - **Booleans/Strings**: The value must be in the
   ground truth's `value` array (e.g., `true` or `1` or `OK
   ` etc).

3.  **Code-Level Root Cause Analysis**:
    - For every discrepancy found, you MUST inspect the
   provided `Decoder_Script`.
    - Pinpoint the likely line(s) of code or logic block
   responsible for the error.
    - Formulate a clear, actionable suggestion for how to
   fix the script.

## Evaluation Rubric

- **1.0 (Perfect)**: All required data points are present
   with correct values. The decoder script is flawless.
- **0.75 (Structurally Imperfect)**: All required data
   points are present and their values are correct. However
   , there are structural errors, such as a data point
   being emitted on the wrong topic. The core data is
```

```
   intact.
- **0.5 (Partially Incorrect)**: One or two non-optional
   data points are **completely missing** from the output,
   or a key value is fundamentally wrong. This indicates a
   significant flaw in the decoder's logic.
- **0.25 (Largely Incorrect)**: Multiple non-optional data
   points are missing. The decoder is failing to extract
   most of the required information.
- **0.0 (Completely Incorrect)**: The output is empty,
   malformed, or has no correlation with the ground truth.

## Input Format

You will receive a JSON object with three keys:
- `Ground_Truth`: The specification of expected device
   behavior.
- `Payloads`: The actual data emitted by the device.
- `Decoder_Script`: The source code JavaScript used to
   parse the raw device payload.
- `Original_Script`: The original source code JavaScript
   before any modifications.

## Output Format

Your final response MUST be a single JSON object. The `
   analysis_and_suggestions` field must contain three
   distinct sections: **Overall Assessment**, **
   Discrepancies**, and **Actionable Suggestions**.
```

Listing A.3: System instruction separated in description, goal, rules, rubric, input, and output.

A.6 Artifact Requirements

A.6.1 Topic Schema (*.schema.json)

- The schema must be written as valid JSON.
- It must include all required schema keys: `$id`, `$schema`, `type`, `processingType`, `topic`, `title`, `properties`, and `required`.
- The topic name must start and end with an alphanumeric character.
- The topic name must consist of at least two alphanumeric characters.
- The topic name can contain an optional separator after the first two alphanumeric characters, followed by at least one alphanumeric character. Valid separators are dots (`.`), colons (`:`), hyphens (`-`), and underscores (`_`).

- Topic name separators cannot be consecutive and cannot be placed at the beginning or the end of the name.
- Schema properties must either be references using the `$ref` keyword or include a `title`, `description`, and `type`.
- Referenced schemas are permitted to override the `title` and `description` attributes.
- Property key names must use camelCase.

A.6.2 Metadata (meta.json)

- The file must be valid JSON.
- It must include specific required keys: `name`, `version`, `manufacturer`, `url`, `description`, `author`, `firmwareVersion`, `availableSensors`, `outputTopics`, `encoding`, and `connectivity`.
- The `firmwareVersion` property must follow a specific format.
- All topics listed in the `outputTopics` property must exist as corresponding topic schemas.

A.6.3 Decoder Script (uplinks.js)

- The script must be valid JavaScript ES11 code.
- External modules cannot be used.
- The script is allowed to use the provided utility functions.
- A `consume()` function with a specific signature must be implemented.
- The script must include `emit()` function calls.
- There cannot be more than 50 `emit()` function calls per script execution.
- The calls must follow a string format with exactly two arguments: the first indicating the emit type and the second being an object with a specific structure.
- The data emitted must match the exact format specified in the corresponding topic schema.
- No `console.log` calls are allowed.
- File handling operations are not allowed.
- The script must decode the data exactly as specified in the provided device decoding logic.

A.7 Test Set Device List

Device Name	Manufacturer	Source
VEGAPULS Air 41	VEGA	(VEGA Grieshaber KG, 2022)
OY1700	Talkpool	(Talkpool, 2019)
EMS Desk	Elsys	(Elsys, 2025)
WT201	Milesight	(Milesight, 2025)
IAM	Decentlab	(Decentlab, 2025)

Table A.1: Devices used for evaluation benchmark testing and refinement loop

A.8 Issues List From Evaluation Benchmark

Issue Description	Agent	Count
Schema definition invalid (must contain `$ref` or `title`, `desc`, `type`)	Schema Agent	20
Data key contains underscore error	Schema Agent	20
Data key pattern mismatch (e.g., single letter 'g')	Schema Agent	10
Invalid `firmwareVersion` format (must match pattern `^V([0-9]+)...`)	Meta Agent	9
Missing required property `firmwareVersion`	Meta Agent	6
Missing required property `topic`	Schema Agent	3
Missing required property `url`	Meta Agent	2
Invalid `connectivity` value (not in allowed list)	Meta Agent	1
Invalid `connectivity` type (must be string)	Meta Agent	1

Table A.2: Common Issues Identified During First Iterative Refinement Cycle

A.9 Issues List From Manual Inspection

Issue Description	**Agent**
Enum values are not capitalized	Schema Agent
Broken schema references (references do not exist in default schema)	Schema Agent
Unnecessary overwriting of properties in schema references	Schema Agent
Device name includes manufacturer name (redundant)	Meta Agent
Metadata (e.g., `batteryVoltage`) incorrectly listed as available sensor	Meta Agent
Firmware version does not default to `v1.0.0`	Meta Agent
Decoder logic components placed inside `consume` function	Decoder Agent
Code formatting violates Prettier standards (e.g., line breaks)	Decoder Agent
Incorrect modification of the payload timestamp	Decoder Agent
Failure to emit individual entries for array/datalog payloads	Decoder Agent
Ignores provided manufacturer reference decoder (reinvents logic)	Decoder Agent
Character encoding issues (e.g., raw Unicode `\u00b5`)	Decoder Agent
Inconsistent coding style (function headers, naming conventions)	Decoder Agent

Table A.3: Issues Identified Through Manual Review of Artifacts

A.10 Notes Case Study Phases

Stage	Manual Notes	LLM-Assisted Notes
Setup	Create new branch, folder, and files.	Create new branch and folder.
Research	Gather relevant documentation and general TTN decoder; review documentation for decoding logic and search for sample payloads for testing.	Gather decoding logic documentation, sample decoder, and sample payloads for testing.
Decoder Script	Adapt existing script; identify and extract all payload segments using manufacturer documentation.	Set up LLM-Assistant, launch artifact creation, test generated code, and fix issues.
Topic Schemas	Identify all possible $ref values and create three topic schemas.	Check all generated topic schemas and adjust required fields.
Testing	Created two unit tests for default and configuration uplink messages found in documentation; adjusted uplink.js for compatibility.	Created unit tests for default and datalog messages.
Metadata	Copied and pasted skeleton meta.json from other device type and adjusted.	Review generated meta.json.
PR	Final check, added image and changelog, and created pull request.	Final check, added image, and created pull request.
PR Comments Fix	Cleaned up logic and removed lifecycle from historic datalog.	Added validation for historic date emission, adjusted a property to an enum, and implemented date mocking in tests.

Table A.4: Detailed Notes from Case Study

A.11 Problem Definition

Note: The following document is the original problem definition PDF.

Investigating and Prototyping an LLM-Based System for the Generation, Evaluation, and Acceleration of IoT Payload Decoder Development

Bachelor Thesis Problem Definition

School of Computer Science and Information Technology Lucerne
University of Applied Sciences and Arts (Switzerland)

Author: Tillo Beffa
Supervisor: Prof. Dr. René Meier

September 2025

Problem Definition

1.1 General Information

Module: Dept I BAA HS25

1.2 Thesis Title

Investigating and Prototyping an LLM-Based System for the Generation, Evaluation, and Acceleration of IoT Payload Decoder Development

1.3 Background and Problem Statement

The Akenza IoT platform serves as a central interface for connecting and managing a wide variety of IoT devices, as well as for processing the data they generate. These devices, whether they are sensors in agriculture, meters in smart buildings, or trackers in logistics, frequently send their measured values and status information in the form of very compact, often in hex or vendor-specific encoded "payloads". This is done to save transmission bandwidth and energy, which is particularly important for mostly battery-powered devices operating over wireless networks like LoRaWAN.

However, for this raw data, which is often incomprehensible to humans and even directly to machines, to become usable for downstream applications such as visualizations in dashboards, alerts, or further analysis on the Akenza platform, it must first be interpreted and converted into a standardized, structured format (such as JSON) [1].

This is precisely where the creation of payload decoders comes in: For users of the Akenza IoT platform, this is an important but often challenging step. These decoders are scripts whose development requires an understanding of specific device data and Akenza's scripting logic. This manual process not only makes decoder creation time-consuming but also prone to errors, which can compromise the quality of subsequent data processing.

1.4 Thesis Goal and Core Objectives

The primary goal of this thesis is to investigate, design, and prototype an assistance system that leverages Large Language Models to optimize and accelerate the creation of IoT payload decoders.

Objective 1: Research and Technology Exploration:

- Conduct a investigation into state-of-the-art LLMs and prompt engineering strategies for domain-specific code generation.
- Explore additional technologies to enhance the LLM's performance, including Retrieval-Augmented Generation with vector databases, real-time information retrieval (e.g., web scraping), and agentic frameworks for orchestrating complex tasks.

Objective 2: Build a Prototype Tool:

- Develop a functional prototype, likely as a command-line interface or a similar standalone tool.
- The tool will be designed to accept device information as input and generate key artifacts, such as the decoder script and its corresponding JSON schema.
- Follow best practices in software development and state of the art LLM integration, ensuring the prototype is modular, maintainable, and well-documented.

Objective 3: Development of a Custom Evaluation Benchmark:

- Design and implement a benchmark to systematically assess the quality of AI-generated decoder scripts.
- Define a clear set of evaluation metrics, focusing on functional correctness, completeness, adherence to Akenzas scripting guidelines and coding best practices.

Objective 4: Iterative Improvement:

- Apply the evaluation benchmark to the prototype's output to generate quantitative performance data.
- Analyze these results to identify weaknesses and opportunities for improvement, creating a feedback loop to iteratively refine the system by optimizing prompts, comparing models, or adjusting the integration of additional tools.
- Investigate and conceptualize a human-in-the-loop approach where user corrections or feedback can be captured to further improve the quality of generated artifacts.

Objective 5: Quantifying Practical Value and Efficiency Gains:

- Establish a performance baseline by measuring the time required for a developer to create decoders manually.
- Conduct a comparative study to measure the time and effort saved when using the prototype, providing a quantitative assessment of the tool's potential impact on developer productivity.

1.5 Proposed Methods and Approach

This Thesis will use a mixed-methods approach, combining qualitative research, design science, and quantitative evaluation to achieve its objectives.

Qualitative Methods

- Systematic Literature Review: To identify and analyze state-of-the-art Large Language Models, prompt engineering strategies, and additional technologies/tools relevant to code generation.
- Exploratory Analysis: To examine existing IoT payload decoders, device datasheets and industry standards, identifying common patterns and structures.

Design Science and Prototyping

- Artifact Creation: The core of the project involves the development of two key artifacts: (1) a functional prototype of the LLM-based assistance tool and (2) a custom benchmark to evaluate its output.

- Iterative Development: The prototype will be built and refined through an iterative cycle of development and testing, guided by feedback from the benchmark.

Quantitative Methods

- Performance Benchmarking: The custom benchmark will be used to conduct automated experiments, generating quantitative data on the performance of the generated decoders based on metrics like functional correctness and completeness.
- Comparative Study: A controlled experiment will be conducted to compare the time required for developers to create decoders with and without the assistance tool. This study will provide quantitative data to assess the tool's impact on productivity.

1.6 Innovation and Academic Challenge

This thesis presents several challenges that align with the business information technology degree:

- **Methodological Challenge:** The primary innovation is the design of a scientific and reproducible methodology to measure and enhance an LLM's performance for a real-world software engineering task.
- **Technical Challenge:** The project requires the integration of state-of-the-art AI technologies into a practical developer tool. The key technical difficulty is building a robust automated benchmark system capable of reliably testing dynamically generated code.
- **Evaluation and Optimization Challenge:** A central academic challenge is to establish a data-driven process for improving the AI system. This extends to translating technical performance metrics (e.g., code correctness) into business value metrics (e.g., time saved), important for assessing the ROI of AI solutions.

1.7 Related and Relevant Work

The rapid advancement of large language models has sparked extensive research in areas such as prompting techniques, autonomous agents, Retrieval-Augmented Generation frameworks, and benchmarking methodologies.

Regarding benchmarking, a key distinction must be made: While general benchmarks like HumanEval [2] assess an LLM's overall code generation capabilities and can inform model selection, they often lack the domain-specific evaluation crucial for specialized applications. This project, focused on generating dynamic code for a specific domain with strict rules, requires a tailored benchmarking approach for which a direct, off-the-shelf solution is currently unavailable. However, general benchmarking principles still offer a valuable foundational understanding.

More applicable research for this project includes frameworks and metrics specifically designed to evaluate generated code and artifacts within highly domain-specific contexts [3, 4, 5, 6, 7, 8].

Research into the effective utilization of LLMs through advanced prompting, tool integration, function calling, agents, and RAG has seen significant advancements. These works will heavily influence the Proof of Concept implementation for this project, including surveys on LLM-based agents [9], prompt engineering [10, 11, 12], and RAG for LLMs [13, 14, 15]. Additionally, research on LLM hallucination is key for ensuring the reliability of generated code [16].

Project Team

Student: Tillo Beffa
Supervisor: Prof. Dr. René Meier

2.1 Client

Company:	Akenza AG
Contact:	Marius Giger
Role:	Head of Software
Address:	Binzmühlestrasse 170A
Postal Code/City:	CH-8050 Zürich
Phone:	[redacted]
E-Mail:	
Website:	`https://akenza.io/`

Bibliography

[1] Datacake Docs, "Lorawan payload decoders." `https://docs.datacake.de/lorawan/payload-decoders`, 2024. Accessed: 13 September 2025.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021.

[3] D. G. Paul, H. Zhu, and I. Bayley, "Benchmarks and metrics for evaluations of code generation: A critical review," 2024. Accessed: 13 September 2025.

[4] X. Gu, M. Chen, Y. Lin, Y. Hu, H. Zhang, C. Wan, Z. Wei, Y. Xu, and J. Wang, "On the effectiveness of large language models in domain-specific code generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, Feb. 2025.

[5] J. Li, G. Li, X. Zhang, Y. Zhao, Y. Dong, Z. Jin, B. Li, F. Huang, and Y. Li, "Evocodebench: An evolving code generation benchmark with domain-specific evaluations," 2024. Accessed: 13 September 2025.

[6] Z. Ye, Z. Yan, J. He, T. Kasriel, K. Yang, and D. Song, "Verina: Benchmarking verifiable code generation," 2025. Accessed: 13 September 2025.

[7] W. Tong and T. Zhang, "Codejudge: Evaluating code generation with large language models," 2024. Accessed: 13 September 2025.

[8] P. Xue, L. Wu, Z. Yang, C. Wang, X. Li, Y. Zhang, J. Li, R. Jin, Y. Pei, Z. Shen, X. Lyu, and J. W. Keung, "Classeval-t: Evaluating large language models in class-level code translation," 2025. Accessed: 13 September 2025.

[9] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The rise and potential of large language model based agents: A survey," 2023. Accessed: 13 September 2025.

[10] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, P. S. Dulepet, S. Vidyadhara, D. Ki, S. Agrawal, C. Pham, G. Kroiz, F. Li, H. Tao, A. Srivastava, H. D. Costa, S. Gupta, M. L. Rogers, I. Goncearenco, G. Sarli, I. Galynker, D. Peskoff, M. Carpuat, J. White, S. Anadkat, A. Hoyle, and P. Resnik, "The prompt report: A systematic survey of prompt engineering techniques," 2025. Accessed: 13 September 2025.

[11] X. Amatriain, "Prompt design and engineering: Introduction and advanced methods," 2024. Accessed: 13 September 2025.

[12] K. Chang, S. Xu, C. Wang, Y. Luo, X. Liu, T. Xiao, and J. Zhu, "Efficient prompting methods for large language models: A survey," 2024. Accessed: 13 September 2025.

[13] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2024. Accessed: 13 September 2025.

[14] S. Zhao, Y. Yang, Z. Wang, Z. He, L. K. Qiu, and L. Qiu, "Retrieval augmented generation (rag) and beyond: A comprehensive survey on how to make your llms use external data more wisely," 2024. Accessed: 13 September 2025.

[15] B. Jin, J. Yoon, J. Han, and S. O. Arik, "Long-context llms meet rag: Overcoming challenges for long inputs in rag," 2024. Accessed: 13 September 2025.

[16] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, p. 1–55, Jan. 2025.

Appendix B

Lists

B.1 List of Figures

2.1 Example of a hexadecimal payload string (Akenza, 2025b) 5
2.2 Example of the decoded payload (Akenza, 2025b) 5
2.3 Akenza Data Processing Pipeline Simplified 7
2.4 Example uplink.js decoder script 8
2.5 Example topic schema (default.schema.json) 9
2.6 Example meta.json for the Netvox R711 device (Akenza, 2024) . . . 9

4.1 Roadmap Bachelor Thesis 21

5.1 Agent workflow of final LLM-Assistant prototype 28
5.2 CLI Startup Menu 31
5.3 CLI Create 31
5.4 CLI Create Input 32
5.5 CLI Stats 32
5.6 Unit Test Results 33
5.7 Benchmark initialization 39
5.8 Execution and contract testing 39
5.9 Syntax validation phase 39
5.10 Semantic scoring phase 39

B.2 List of Tables

3.1 State-of-the-art Research Plan Topics 16

5.1 Hierarchical Overview of Benchmark Metrics and Weights 38
5.2 Meta-Evaluation Results: Pro vs. Flash Model 40
5.3 Benchmark Scores Before Refinement 41
5.4 Benchmark Scores After Refinement 42
5.5 Gemini Model Pricing as of 22.12.2025 (Google, 2025b) 42

6.1 Time Tracking Comparison: Manual vs. LLM-Assisted Creation . . 45

A.1 Devices used for evaluation benchmark testing and refinement loop 63
A.2 Common Issues Identified During First Iterative Refinement Cycle . 63
A.3 Issues Identified Through Manual Review of Artifacts 64
A.4 Detailed Notes from Case Study 65

B.1 Overview of AI Tool Usage 73

B.3 AI-Usage

Tool	**Date**	**Reason for Use**	**Position of Use**
Grammarly	Sep - Dec 2025	Identifying and correcting grammatical and spelling errors during the writing process.	Entire Document
Google Gemini	Dec 2025	Proofreading for grammatical and spelling errors; assistance with writing LaTeX code.	Entire Document
Google DeepSearch	Sep 2025	Assistance in finding relevant sources for the literature review.	Chapter 2

Table B.1: Overview of AI Tool Usage

References

Ajv. (2025). *Ajv JSON Schema Validator.* https://ajv.js.org/. Retrieved 2025-12-22, from https://ajv.js.org/

Akenza. (2024). *meta.json for netvox r711 device type.* https://github.com/akenza-io/device-type-library/blob/d43d010b/types/netvox/r711/meta.json. (Commit: d43d010b)

Akenza. (2025a). *Akenza Device Type Library.* https://github.com/akenza-io/device-type-library/tree/main. Retrieved 2025-12-22, from https://github.com/akenza-io/device-type-library/tree/main

Akenza. (2025b). *Akenza Device Type Library: Netvox R712.* https://github.com/akenza-io/device-type-library/tree/main/types/netvox/R712. Retrieved 2025-12-21, from https://github.com/akenza-io/device-type-library/tree/main/types/netvox/R712

Akenza. (2025c). *Akenza Scripting Documentation.* https://docs.akenza.io/akenza.io/get-started/reference/scripting. Retrieved 2025-12-19, from https://docs.akenza.io/akenza.io/get-started/reference/scripting

Akenza. (2025d). *Akenza Scripting: Topic Validation.* https://docs.akenza.io/akenza.io/get-started/reference/scripting#topic-validation. Retrieved 2025-12-19, from https://docs.akenza.io/akenza.io/get-started/reference/scripting#topic-validation

Akenza. (2025e). *Akenza System Overview and Data Flow.* https://docs.akenza.io/akenza.io#systemoverview-dataflow. Retrieved 2025-12-19, from https://docs.akenza.io/akenza.io#systemoverview-dataflow

Akenza. (2025f). *Custom device types.* https://docs.akenza.io/akenza.io/get-started/your-data-flow/device-type/device-type. Retrieved 2025-10-31, from https://docs.akenza.io/akenza.io/get-started/your-data-flow/device-type/device-type

Astral. (2025). *uv Documentation.* https://docs.astral.sh/uv/. Retrieved 2025-12-21, from https://docs.astral.sh/uv/

Casals, L., Gomez, C., & Vidal, R. (2023). *Understanding the impact of packet size on the energy efficiency of lorawan.* Published in Journal of Communications and Networks, vol. 25, no. 6, pp. 814-824. doi: 10.23919/JCN.2023.000039

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., ... Zaremba, W. (2021). *Evaluating large language models trained on code.* Retrieved from https://arxiv.org/abs/2107.03374

Crupi, G., Tufano, R., Velasco, A., Mastropaolo, A., Poshyvanyk, D., & Bavota, G. (2025). *On the effectiveness of llm-as-a-judge for code generation and summarization.* Retrieved from https://arxiv.org/abs/2507.16587

Decentlab. (2025). *Indoor Ambiance Monitor including CO_2, TVOC and Motion Sensor for LoRaWan.* https://www.decentlab.com/products/indoor-ambiance-monitor-including-co2-tvoc-and-motion-sensor-for-lorawan. Retrieved 2025-12-22, from https://www.decentlab.com/products/indoor-ambiance-monitor-including-co2-tvoc-and-motion-sensor-for-lorawan

Dragino. (2025a). *Dds04-lb/ls lorawan distance sensor.* https://www.dragino.com/products/distance-level-sensor/item/280-dds04-lb-ls.html. Retrieved 2025-12-23, from https://www.dragino.com/products/distance-level-sensor/item/280-dds04-lb-ls.html

Dragino. (2025b). *Dds20-lb/ls lorawan distance sensor.* https://www.dragino.com/products/distance-level-sensor/item/273-dds20-lb-ls.html. Retrieved 2025-12-23, from https://www.dragino.com/products/distance-level-sensor/item/273-dds20-lb-ls.html

Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., ... Lou, Y. (2023). *Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation.* Retrieved from https://arxiv.org/abs/2308.01861

Eastgate Software. (2025). *Top Python AI Agent frameworks and no-code tools for 2025.* https://eastgate-software.com/top-python-ai-agent-frameworks-and-no-code-tools-for-2025/. Retrieved 2025-12-19, from https://eastgate-software.com/top-python-ai-agent-frameworks-and-no-code-tools-for-2025/

Eight04. (2023). *node-vm2 (PyPI Project).* https://pypi.org/project/node-vm2/. Retrieved 2025-12-21, from https://pypi.org/project/node-vm2/

Elsys. (2025). *EMS Desk: LoRaWan Desk Occupancy and Indoor Environment Sensor.* https://www.elsys.se/en/ems-desk/. Retrieved 2025-12-22, from https://www.elsys.se/en/ems-desk/

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... Wang, H. (2024). *Retrieval-augmented generation for large language models: A survey.* Retrieved from https://arxiv.org/abs/2312.10997

Google. (2025a). *Agent development kit documentation.* https://google.github.io/adk-docs/.

Google. (2025b). *Gemini API Pricing.* https://ai.google.dev/gemini-api/docs/pricing. Retrieved 2025-12-22, from https://ai.google.dev/gemini-api/docs/pricing

GraalVM Project. (2025). *GraalVM JavaScript and Node.js Reference.* https://www.graalvm.org/latest/reference-manual/js/. Retrieved 2025-12-19, from https://www.graalvm.org/latest/reference-manual/js/

Gu, X., Chen, M., Lin, Y., Hu, Y., Zhang, H., Wan, C., ... Wang, J. (2025). On the effectiveness of large language models in domain-specific code generation. *ACM Trans. Softw. Eng. Methodol.*, *34*(3). Retrieved from https://doi.org/10.1145/3697012 doi: 10.1145/3697012

Gutknecht, J. (2025). *Interview on iot decoder creation process.* Personal Interview. (See Appendix A.1.1 for the full transcript)

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., ... Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, *43*(2), 1–55. Retrieved from http://dx.doi.org/10.1145/3703155 doi: 10.1145/3703155

Jiang, J., Wang, F., Shen, J., Kim, S., & Kim, S. (2024). *A survey on large language models for code generation.* Retrieved from https://arxiv.org/abs/2406.00515

JSON Schema. (2017). *JSON Schema Draft 07.* https://json-schema.org/draft-07#draft-07. Retrieved 2025-12-19, from https://json-schema.org/draft-07#draft-07

Leibbrandt, A., Alexis. (2021). *How to decode the payload of your device.* https://akenza.io/blog/payload-decoding. Retrieved 2025-10-31, from https://akenza.io/blog/payload-decoding

Li, J., Li, G., Zhang, X., Zhao, Y., Dong, Y., Jin, Z., ... Li, Y. (2024). *Evocodebench: An evolving code generation benchmark with domain-specific evaluations.* Retrieved from https://arxiv.org/abs/2410.22821

LLM Stats. (2025). *Gemini 2.5 Flash vs Gemini 2.5 Pro - LLM Stats.* https://llm-stats.com/models/compare/gemini-2.5-flash-vs-gemini-2.5-pro. Retrieved 2025-12-22, from https://llm-stats.com/models/compare/gemini-2.5-flash-vs-gemini-2.5-pro

LoRa Alliance. (2020). *Lorawan® l2 1.0.4 specification.* https://resources.lora-alliance.org/technical-specifications/ts001-1-0-4-lorawan-l2-1-0-4-specification.

Microsoft. (2020). *CodeBLEU: Evaluator for Code-to-Code Translation.* https://github.com/microsoft/CodeXGLUE/tree/main/Code-Code/code-to-code-trans/evaluator/CodeBLEU. Retrieved 2025-12-22, from https://github.com/microsoft/CodeXGLUE/tree/main/Code-Code/code-to-code-trans/evaluator/CodeBLEU

Milesight. (2025). *WT201 Smart LoRaWan Thermostat.* https://www.milesight.com/iot/product/lorawan-sensor/wt201. Retrieved 2025-12-21, from https://www.milesight.com/iot/product/lorawan-sensor/wt201

Netvox. (2025). *R712 - Wireless Outdoor Temperature Humidity Sensor.* http://www.netvox.com.tw/product.asp?pro=R712. Retrieved 2025-12-21, from http://www.netvox.com.tw/product.asp?pro=R712

Noura, M., Atiquzzaman, M., & Gaedke, M. (2018). *Interoperability in internet of things: Taxonomies and open challenges.* Retrieved 2025-12-19, from https://doi.org/10.1007/s11036-018-1089-9 doi: 10.1007/s11036-018-1089-9

OpenAI. (2024). *Gpt-5 prompting guide.* https://cookbook.openai.com/examples/gpt-5/gpt-5_prompting_guide.

Pallets Projects. (2025). *Click Documentation.* https://click.palletsprojects.com/en/stable/. Retrieved 2025-12-21, from https://click.palletsprojects.com/en/stable/

Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). *BLEU: a method for automatic evaluation of machine translation.* Retrieved from https://aclanthology.org/P02-1040.pdf (Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), Philadelphia, pp. 311-318)

Paul, D. G., Zhu, H., & Bayley, I. (2024). *Benchmarks and metrics for evaluations of code generation: A critical review.* Retrieved from https://arxiv.org/abs/2406.12655

Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., ... Ma, S. (2020). *Codebleu: a method for automatic evaluation of code synthesis.* Retrieved from https://arxiv.org/abs/2009.10297

Rüegger, J. (2025). *Interview regarding the evolution of genio and agentic systems.* Personal Interview. (See Appendix A.1.2 for the full transcript)

Sankalp. (2025). *How prompt caching works - Paged Attention and Automatic Prefix Caching plus practical tips.* https://sankalp.bearblog.dev/how-prompt-caching-works/. Retrieved 2025-12-21, from https://sankalp.bearblog.dev/how-prompt-caching-works/

Schulhoff, S., Ilie, M., Balepur, N., Kahadze, K., Liu, A., Si, C., ... Resnik, P. (2025). *The prompt report: A systematic survey of prompt engineering techniques.* Retrieved from https://arxiv.org/abs/2406.06608

Stryker, C. (2024). *What are large language models (LLMs)?* https://www.ibm.com/think/topics/large-language-models. (Part of the IBM Think publication)

Talkpool. (2019). *Talkpool's New IoT Indoor Air Quality Sensors: OY1700 Particulate Matter Sensor.* https://talkpool.com/talkpools-2-new-iot-indoor-air-quality-sensors/. Retrieved 2025-12-22, from https://talkpool.com/talkpools-2-new-iot-indoor-air-quality-sensors/

Tektelic. (n.d.). *How often can lorawan devices send data?* https://tektelic.com/expertise/how-often-can-lorawan-devices-send-data/. Retrieved 2025-10-31, from https://tektelic.com/expertise/how-often-can-lorawan-devices-send-data/

Tong, W., & Zhang, T. (2024). *Codejudge: Evaluating code generation with large language models.* Retrieved from https://arxiv.org/abs/2410.02184

VEGA Grieshaber KG. (2022). *VEGAPULS Air 41: Autarkic Radar Sensor with Measured Value Transmission.* https://www.vega.com/de-ch/produkte/produktkatalog/fuellstand/radar/vegapuls-air-41. Retrieved 2025-12-22, from https://www.vega.com/de-ch/produkte/produktkatalog/fuellstand/radar/vegapuls-air-41

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., ... Gui, T. (2023). *The rise and potential of large language model based agents: A survey.* Retrieved from https://arxiv.org/abs/2309.07864

Yang, C., Shi, Y., Ma, Q., Liu, M. X., Kästner, C., & Wu, T. (2025). *What prompts don't say: Understanding and managing underspecification in llm prompts.* Retrieved from https://arxiv.org/abs/2505.13360